Guida al progetto «Harden a C application with ASPIS»

Tutto quello che c'è da sapere per capire, rifare e presentare il progetto

Per Alberto Rotolo e Simone Rota

Linguaggi Formali e Compilatori, A.A. 2026/27, Politecnico di Milano

---

Indice

Come leggere questa guida
Parte I. I concetti
1. Il progetto in cinque minuti
   1.1 Cosa chiede la proposta
   1.2 Cosa è stato fatto
   1.3 Il messaggio principale dei risultati
   1.4 Il repository
   1.5 Una nota sulla dimensione del progetto
2. Compilatori e LLVM: il minimo indispensabile
   2.1 Cosa fa un compilatore
   2.2 LLVM
   2.3 L'IR di LLVM
   2.4 Basic block e grafo di controllo del flusso
   2.5 I pass
   2.6 I puntatori opachi e la deduzione dei tipi
   2.7 Un minimo di x86-64
3. Guasti transitori e tolleranza ai guasti via software
   3.1 Soft error
   3.2 Guasto, errore, fallimento
   3.3 Gli esiti di un guasto
   3.4 Protezione hardware e protezione software
   3.5 Modelli di guasto
   3.6 Fault injection
4. Le tecniche implementate da ASPIS
   4.1 EDDI: duplicare le istruzioni
   4.2 Il controllo del flusso a firme
   4.3 CFCSS
   4.4 RASM
   4.5 Combinare dati e flusso
Parte II. Il lavoro fatto
5. ASPIS in pratica
   5.1 Il repository di ASPIS
   5.2 Compilare ASPIS
   5.3 Usare ASPIS
   5.4 La pipeline di `aspis.sh`

5.5 FuncRetToRef
5.6 Come EDDI trasforma il codice
5.7 Le altre parti
5.8 Come si vede RASM nel codice
6. Le quattro applicazioni
6.1 Perché queste e con quali regole
6.2 `aes128.c`: cifratura AES-128
6.3 `fft.c`: trasformata di Fourier
6.4 `huffman.c`: compressione di Huffman
6.5 `pid_control.c`: controllore PID
7. I test di compatibilità
7.1 Il sistema di test di ASPIS
7.2 Cosa abbiamo fatto
7.3 I risultati
8. I due bug trovati in ASPIS
8.1 Come si cerca un bug in un pass
8.2 Il bug di EDDI: la copia dopo le chiamate di libreria
8.3 Il bug di RASM: i blocchi di EDDI non venivano controllati
9. Iniettare i guasti con gdb e valgrind
9.1 La golden run
9.2 Scegliere l'istante
9.3 Un esperimento con gdb
9.4 Classificare l'esito
9.5 Quanto costa un esperimento
9.6 Cosa non facciamo
10. La campagna e la statistica
10.1 Il file di configurazione
10.2 Cosa fa `campaign.py`
10.3 Percentuali e intervalli di confidenza
10.4 La media geometrica
10.5 Gli script di analisi
11. I risultati, spiegati
11.1 Il costo delle protezioni
11.2 Bit-flip nei registri
11.3 Salti condizionali invertiti
11.4 Il bug di RASM nei numeri
11.5 Per applicazione
11.6 Il messaggio in tre frasi
Parte III. Pratica ed esame
12. L'integrazione in ASPIS
13. Rifare tutto da zero
14. Mappa dei file
15. Cosa è stato tolto (le versioni precedenti)
16. La chiamata con i supervisori
16.1 Parte 1: il progetto (slide 1-2)
16.2 Parte 2: le applicazioni e i test (slide 3-6)
16.3 Parte 3: la fault injection (slide 7-13)
16.4 Parte 4: integrazione, limiti, IA (slide 14-17)
16.5 Dopo la chiamata

17. Domande probabili all’orale
18. Glossario
Appendice. La verifica dei fatti
A.1 Il metodo
A.2 La fault injection
A.3 I test di compatibilità
A.4 Le applicazioni
A.5 Il bug di EDDI
A.6 La correzione di RASM e le patch
A.7 Fonti e citazioni
A.8 I tempi
A.9 Cosa è stato corretto
A.10 Cosa resta fuori

Come leggere questa guida

Questa guida spiega **tutto** il progetto, partendo da zero. Si assume solo quello che uno studente di ingegneria informatica conosce già: il linguaggio C, le basi di architettura dei calcolatori (registri, istruzioni, memoria) e di sistemi operativi (processi, segnali, `fork` / `exec`). Non si assume nessuna conoscenza del corso di Linguaggi Formali e Compilatori, di LLVM o di tolleranza ai guasti.

La guida è divisa in tre parti:

- **Parte I, i concetti** (capitoli 1-4): cosa chiede il progetto, cos'è un compilatore e com'è fatto LLVM, cosa sono i guasti transitori e come si proteggono i programmi via software, quali tecniche implementa ASPIS.
- **Parte II, il lavoro fatto** (capitoli 5-11): come si usa ASPIS, le quattro applicazioni, i test, i due bug trovati in ASPIS, come iniettiamo i guasti con gdb e valgrind, la campagna di fault injection e i suoi risultati.
- **Parte III, pratica ed esame** (capitoli 12-18): come integrare il lavoro in ASPIS, come rifare tutti gli esperimenti, la mappa dei file, cosa è stato tolto dalle versioni precedenti, cosa dire slide per slide nella chiamata con i supervisori, le domande probabili all'orale con le risposte e un glossario.

Il consiglio è leggere i capitoli nell'ordine, con il repository aperto accanto: ogni volta che si parla di un file, il percorso è scritto in `questo formato`, relativo alla cartella `ASPIS-Project/` del repository (salvo dove indicato diversamente).

Il report ufficiale del progetto, quello che leggeranno i supervisori, è `ASPIS-Project/report/report.pdf` ed è in inglese. Questa guida ne segue la struttura, ma spiega molto di più: il perché delle scelte, i meccanismi interni e i dettagli del codice. Dove serve, riporta i termini inglesi usati nel report.

Parte I. I concetti

1. Il progetto in cinque minuti

1.1 Cosa chiede la proposta

Il progetto «Harden a C application with ASPIS» (supervisori Davide Baroffio ed Emilio Corigliano, del gruppo HEAPLab) ha tre obiettivi:

1. **proteggere applicazioni C con ASPIS**, un insieme di trasformazioni del compilatore LLVM che rendono un programma capace di accorgersi dei guasti hardware;
2. **verificare la protezione con la fault injection**, cioè iniettando guasti artificiali nel programma in esecuzione e osservando cosa succede;
3. **integrare le applicazioni nel branch principale di ASPIS** come test di regressione, così che ogni modifica futura di ASPIS venga provata anche su di esse.

I supervisori hanno anche scritto (file `Buongiorno,.txt` nella radice del repository) che si può usare l'intelligenza artificiale, anche per tutto il progetto, a patto di dichiarare nel report se e per cosa la si è usata, che il numero di programmi da proteggere dipende dalla loro complessità (e se ne può discutere con loro), e che quando saremo pronti ci sarà una chiamata in cui spiegare loro il lavoro; il report servirà come riferimento per la valutazione.

1.2 Cosa è stato fatto

- **Quattro applicazioni** scritte apposta, piccole e capaci di controllare da sole il proprio risultato: cifratura AES-128, trasformata di Fourier (FFT), compressione di Huffman, controllo PID della velocità di un motore.
- **Test di compatibilità** con il sistema di test ufficiale di ASPIS, che compila ogni programma con 55 combinazioni di tecniche. Sul branch principale di ASPIS la prima versione delle applicazioni funzionava solo in 100 delle 220 combinazioni: con EDDI, sEDDI e FDSC (le tecniche che duplicano tutto il programma) le quattro applicazioni andavano in crash.
- **Due bug di ASPIS trovati.** Il primo, in EDDI, causava i crash: lo abbiamo isolato in un programma di cinque righe, descritto in un *bug report* ed evitato nelle applicazioni, che ora funzionano in tutte le 220 combinazioni. Il secondo, in RASM, è stato scoperto grazie alla fault injection e lo abbiamo corretto cambiando una riga; con la correzione i 48 test che ASPIS aveva già danno gli stessi risultati di prima.
- **L'iniezione dei guasti con gdb**, il debugger: uno script di poche decine di righe ferma il programma in un istante casuale e inverte un bit di un registro oppure fa andare un salto condizionale dalla parte sbagliata; valgrind conta le istruzioni eseguite, per scegliere l'istante.
- **Una campagna di fault injection** di 28.000 esperimenti: 4 applicazioni, 6 configurazioni di protezione, 2 modelli di guasto, 500 guasti per combinazione, più 4000 esperimenti per misurare l'effetto del bug di RASM.
- **Due patch** pronte per il repository ufficiale di ASPIS (la correzione di RASM e l'aggiunta dei test) e il bug report di EDDI, non ancora inviati.
- **Il report** in inglese, con la dichiarazione sull'uso dell'AI.

1.3 Il messaggio principale dei risultati

- **EDDI** (duplicazione dei dati) riduce drasticamente i risultati sbagliati non rilevati causati da bit-flip nei registri, ma quasi quadruplica il costo: il programma esegue circa 3,7 volte le istruzioni.
- EDDI **non vede** un salto condizionale andato dalla parte sbagliata, perché le due copie del calcolo seguono lo stesso salto. Lo vedono invece le tecniche di **controllo del flusso a firme**, ma non tutte: **RASM** rileva quasi tutti i salti invertiti, **CFCSS** no, perché verifica solo che il salto vada in un blocco "ammesso", non in quello giusto.
- Combinare EDDI con RASM avrebbe dovuto dare il meglio dei due mondi, ma la fault injection ha mostrato che la combinazione rilevava i salti invertiti **peggio** di RASM da solo: era il bug di RASM, ora corretto.

I numeri precisi sono nel capitolo 11.

1.4 Il repository

Cartella o file	Contenuto
`ASPIS-Project/benchmarks/`	le quattro applicazioni
`ASPIS-Project/aspis-integration/`	le due patch per ASPIS, le voci dei test, il bug report di EDDI e il suo riproduttore
`ASPIS-Project/fault_injection/`	l'esperimento con gdb (`gdb_inject.py`), la campagna (`campaign.py`, `campaign.toml`), l'analisi (`analyze.py`, `sdc_sites.py`, `check_jumps.py`)
`ASPIS-Project/results/`	i log dei test e i risultati della campagna
`ASPIS-Project/report/`	il report (`report.md`, `report.pdf`) e lo script che lo genera
`guida/`	questa guida (fuori dalla cartella del progetto)

1.5 Una nota sulla dimensione del progetto

La prima versione del progetto era molto più grande: 22 programmi di test, 23 bug corretti in ASPIS, un iniettore più sofisticato e una campagna di 240.900 esperimenti. Era troppo per un progetto che vale il 30% di un esame da 5 CFU. Una seconda versione aveva già le quattro applicazioni attuali, ma correggeva ancora due bug dentro EDDI, con modifiche al codice C++ di LLVM difficili da studiare e da difendere all'orale. Una terza versione teneva solo la correzione di RASM, che è una riga, ed evitava il bug di EDDI nelle applicazioni, ma usava ancora un iniettore di guasti scritto da noi in C, basato direttamente su `ptrace`. La versione attuale inietta i guasti con gdb e valgrind, strumenti che si usano già nei corsi. Le versioni precedenti non sono andate perse: sono nella storia di git, ai commit `e1d7e8b`, `7074cd6` e `0609127`. Il capitolo 15 riassume cosa contenevano.

2. Compilatori e LLVM: il minimo indispensabile

2.1 Cosa fa un compilatore

Un compilatore traduce un programma da un linguaggio sorgente (per noi il C) a codice macchina. Lo fa in fasi successive:

1. **analisi lessicale**: il testo viene spezzato in *token* (parole chiave, identificatori, numeri, operatori). È qui che entrano in gioco le espressioni regolari e gli automi a stati finiti, la parte "linguaggi formali" del corso;
2. **analisi sintattica** (*parsing*): i token vengono organizzati in un albero sintattico secondo una grammatica libera dal contesto;
3. **analisi semantica**: si controllano tipi, dichiarazioni, regole del linguaggio;
4. **generazione della rappresentazione intermedia** (*IR, intermediate representation*): il programma viene tradotto in un linguaggio più semplice e regolare, indipendente sia dal C sia dalla macchina;
5. **ottimizzazione**: trasformazioni dell'IR che lo rendono più veloce o più piccolo senza cambiarne il significato;
6. **generazione del codice**: l'IR viene tradotto nelle istruzioni del processore (per noi x86-64), con l'allocazione dei registri.

Le fasi 1-4 formano il **front-end** (dipende dal linguaggio sorgente), la 5 il **middle-end** (non dipende né dal sorgente né dalla macchina), la 6 il **back-end** (dipende dalla macchina). Il vantaggio di un IR comune è che si scrive una trasformazione una volta sola e funziona per tutti i linguaggi e tutte le architetture.

ASPIS lavora nel middle-end: le sue protezioni sono **trasformazioni dell'IR**. Non tocca né il parsing del C né la generazione del codice x86.

2.2 LLVM

LLVM è l'infrastruttura di compilazione più usata nella ricerca e in molti prodotti (Apple, Google, Rust, Swift usano LLVM). I programmi che ci interessano sono:

Programma	Cosa fa
`clang`	front-end per C e C++: produce l'IR (con `-S -emit-llvm`) oppure direttamente l'eseguibile
`opt`	esegue dei *pass* (trasformazioni o analisi) su un file IR e scrive l'IR trasformato
`llvm-link`	unisce più file IR in un unico modulo
`clang` su un file `.ll`	fa da back-end: traduce l'IR in codice macchina e collega l'eseguibile
`llvm-size`	stampa la dimensione delle sezioni di un eseguibile (`.text` è il codice)

Il file IR in forma testuale ha estensione `.ll`. Si può leggere, ed è molto utile farlo: quando ASPIS ha un problema, il primo passo è guardare l'IR che produce.

Noi usiamo **LLVM 21.1.8**, la versione richiesta da ASPIS, scaricata già compilata dal sito di LLVM.

2.3 L'IR di LLVM

Un file IR contiene un **modulo**, cioè un insieme di variabili globali e di funzioni. Ogni funzione è fatta di **basic block** (blocchi di base), ciascuno con un'etichetta, e ogni blocco è una sequenza di **istruzioni**.

L'IR è in forma **SSA** (*static single assignment*): ogni valore ha un nome (`%a`, `%0`, `%add` ...) e viene assegnato **una sola volta**. Le variabili del C che cambiano valore non diventano valori SSA, ma celle di memoria sullo stack, create

con `alloca` e lette e scritte con `load` e `store`. Questo è esattamente quello che fa `clang` senza ottimizzazioni (`-O0`), che è il modo in cui lo usa ASPIS.

Esempio. Questa funzione C:

```c
int sum(int *a, int n) {
  int s = 0;
  for (int i = 0; i < n; i++)
    s += a[i];
  return s;
}
```

diventa (con `clang -S -emit-llvm -O0`, tolti alcuni attributi):

```
define i32 @sum(ptr %a, i32 %n) {
entry:
  %a.addr = alloca ptr               ; cella sullo stack per il parametro a
  %n.addr = alloca i32               ; ... per n
  %s = alloca i32                    ; variabile locale s
  %i = alloca i32                    ; variabile locale i
  store ptr %a, ptr %a.addr          ; copia i parametri nelle celle
  store i32 %n, ptr %n.addr
  store i32 0, ptr %s                ; s = 0
  store i32 0, ptr %i                ; i = 0
  br label %for.cond                 ; salto incondizionato

for.cond:                            ; condizione del ciclo
  %0 = load i32, ptr %i
  %1 = load i32, ptr %n.addr
  %cmp = icmp slt i32 %0, %1         ; i < n ? (slt = signed less than)
  br i1 %cmp, label %for.body, label %for.end   ; salto condizionale

for.body:                            ; s += a[i]
  %2 = load ptr, ptr %a.addr
  %3 = load i32, ptr %i
  %idxprom = sext i32 %3 to i64      ; estensione di segno a 64 bit
  %arrayidx = getelementptr inbounds i32, ptr %2, i64 %idxprom   ; &a[i]
  %4 = load i32, ptr %arrayidx       ; a[i]
  %5 = load i32, ptr %s
  %add = add nsw i32 %5, %4
  store i32 %add, ptr %s
  br label %for.inc

for.inc:                             ; i++
  %6 = load i32, ptr %i
  %inc = add nsw i32 %6, 1
  store i32 %inc, ptr %i
  br label %for.cond

for.end:
  %7 = load i32, ptr %s
  ret i32 %7                         ; return s
}
```

Le istruzioni da conoscere:

Istruzione	Significato
`alloca T`	riserva sullo stack lo spazio per un valore di tipo `T`, restituisce il puntatore
`load T, ptr p`	legge un valore di tipo `T` dall'indirizzo `p`
`store T v, ptr p`	scrive `v` all'indirizzo `p`
`getelementptr` (GEP)	calcola un indirizzo (elemento di un array, campo di una struct) senza leggere la memoria
`add`, `sub`, `mul`, `xor`, …	aritmetica intera; `fadd`, `fmul`, … per i numeri in virgola mobile
`icmp`, `fcmp`	confronti (interi e in virgola mobile): danno un `i1`, cioè un booleano
`br i1 c, label A, label B`	salto condizionale: va in `A` se `c` è vero, altrimenti in `B`
`br label A`	salto incondizionato
`select i1 c, T x, T y`	`c ? x : y` senza salti
`call`	chiamata di funzione
`ret`	ritorno dalla funzione
`phi`	sceglie un valore a seconda del blocco da cui si è arrivati (con `-O0` è rara)

I tipi: `i1`, `i8`, `i32`, `i64` sono interi di 1, 8, 32, 64 bit; `float`, `double`; `ptr` è un puntatore; poi ci sono struct (`{ i32, float }`) e array (`[16 x i8]`). I nomi che cominciano con `@` sono globali (funzioni e variabili globali), quelli con `%` sono locali alla funzione.

ASPIS non usa i nomi leggibili come `%a.addr`: `clang` di default li scarta e numera i valori (`%0`, `%1`, …). Il significato non cambia.

2.4 Basic block e grafo di controllo del flusso

Un **basic block** è una sequenza di istruzioni senza salti al suo interno: si entra solo dall'inizio e si esce solo dalla fine. L'ultima istruzione è un *terminatore* (`br`, `ret`, `unreachable`, …), che dice dove si va dopo.

I blocchi di una funzione e i salti tra di loro formano il **grafo di controllo del flusso** (CFG, *control-flow graph*). Per `sum`:



Un'esecuzione corretta percorre sempre archi di questo grafo. Le tecniche di *control-flow checking* (capitolo 4) controllano proprio questo: che il programma segua gli archi giusti.

2.5 I pass

Un **pass** è un pezzo di codice che analizza o trasforma l'IR: per esempio `simplifycfg` semplifica il grafo dei blocchi, `lower-switch` trasforma gli `switch` in catene di `br`. Si eseguono con `opt`:

```
opt --passes="lower-switch" in.ll -o out.ll
```

Un pass può stare fuori dal codice di LLVM, in una libreria condivisa (un *plugin*) caricata al momento:

```
opt -load-pass-plugin=build/passes/libEDDI.so --passes="eddi-verify" in.ll -o out.ll
```

Le protezioni di ASPIS sono pass di questo tipo (*out-of-tree*): sono scritte in C++ usando le classi di LLVM (`Module`, `Function`, `BasicBlock`, `Instruction`, `IRBuilder` per creare nuove istruzioni) e compilate in file `.so`.

2.6 I puntatori opachi e la deduzione dei tipi

Nelle versioni recenti di LLVM i puntatori sono **opachi**: il tipo `ptr` non dice a cosa punta. Una `load i32, ptr %p` dice che in quel punto si legge un `i32`, ma `%p` da solo non lo dice. Questo crea un problema ad ASPIS: per confrontare due puntatori deve confrontare i valori puntati, e quindi deve sapere di che tipo sono. Per questo ASPIS usa una libreria esterna, **TypeDeductionAnalysis** (TDA), che ricostruisce il tipo puntato guardando come il puntatore viene usato.

2.7 Un minimo di x86-64

I guasti si iniettano nel programma già compilato, quindi serve qualche nozione di x86-64:

- **16 registri generali** a 64 bit: `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp`, `r8` … `r15`. `rsp` è il puntatore alla cima dello stack, `rbp` di solito punta al record di attivazione (*frame*) della funzione corrente;
- `rip`, il *program counter*: l'indirizzo della prossima istruzione;
- `rflags`, il registro dei flag, impostato dalle istruzioni aritmetiche e di confronto: `ZF` (risultato zero), `SF` (segno), `CF` (riporto), `OF` (overflow), `PF` (parità);
- i **salti condizionali** (`jcc`: `je`, `jne`, `jl`, `jge`, …) saltano o meno a seconda dei flag. Sono codificati come `0x70`-`0x7f` seguito da uno spostamento di 8 bit, oppure `0x0f 0x80`-`0x8f` seguito da uno spostamento di 32 bit. I 4 bit bassi dell'opcode sono il *condition code* (`cc`);
- con `-O0` ogni variabile vive sullo stack, a un indirizzo come `[rbp - 20]`.

Il ciclo di `sum` compilato con `-O0` (sintassi Intel):

```
.LBB0_1:                                ; for.cond
    mov  eax, dword ptr [rbp - 20]      ; eax = i
    cmp  eax, dword ptr [rbp - 12]      ; confronta i con n (imposta i flag)
    jge  .LBB0_4                        ; se i >= n esce dal ciclo
    mov  rax, qword ptr [rbp - 8]       ; rax = a
    movsxd rcx, dword ptr [rbp - 20]    ; rcx = i (esteso a 64 bit)
    mov  eax, dword ptr [rax + 4*rcx]   ; eax = a[i]
    add  eax, dword ptr [rbp - 16]      ; eax += s
    mov  dword ptr [rbp - 16], eax      ; s = eax
    ...                                 ; i++
    jmp  .LBB0_1
.LBB0_4:
```

Un bit-flip in `rcx` subito prima di `mov eax, dword ptr [rax + 4*rcx]` fa leggere un elemento sbagliato (o un indirizzo non valido, e allora il programma va in crash). Invertire il `jge` fa uscire dal ciclo troppo presto o eseguire un'iterazione di troppo. Sono esattamente i due modelli di guasto del progetto.

3. Guasti transitori e tolleranza ai guasti via software

3.1 Soft error

Una particella (un neutrone dei raggi cosmici, una particella alfa emessa dai materiali del chip) che colpisce un transistor può depositare abbastanza carica da cambiare il valore di un bit: un registro, una cella di memoria, un latch della pipeline. Il componente non si rompe: al prossimo accesso funziona di nuovo. Si parla di **guasto transitorio**, *soft error* o *SEU* (*single event upset*).

Il problema è serio nello spazio e in alta quota (avionica, satelliti) ma anche a terra nei sistemi critici (automotive, ferroviario, medicale), e cresce con la miniaturizzazione: transistor più piccoli e tensioni più basse significano che basta meno carica per invertire un bit.

3.2 Guasto, errore, fallimento

La terminologia classica distingue tre livelli:

- **guasto** (*fault*): la causa fisica, per esempio il bit invertito;
- **errore** (*error*): lo stato del sistema diventato sbagliato, per esempio un valore errato in un registro che verrà usato;
- **fallimento** (*failure*): il servizio fornito è sbagliato, per esempio il programma stampa un risultato errato.

Non tutti i guasti diventano errori (il registro poteva non contenere nulla di utile) e non tutti gli errori diventano fallimenti.

3.3 Gli esiti di un guasto

Quando si inietta un guasto in un programma, l'esecuzione può finire in cinque modi, che ritroveremo in tutti i risultati:

Esito	Significato
masked (mascherato)	il programma termina normalmente con l'output corretto: il guasto non ha avuto effetti (registro non usato, valore sovrascritto, differenza irrilevante)
detected (rilevato)	un controllo inserito dalla protezione si è accorto del guasto e ha chiamato il gestore d'errore
crash	il programma è stato ucciso da un segnale, tipicamente `SIGSEGV` (accesso a un indirizzo non valido)
hang (blocco)	il programma non termina (ciclo infinito) entro un tempo limite
SDC (*silent data corruption*)	il programma termina "normalmente" ma con un risultato sbagliato

L'esito peggiore è la **SDC**: nessuno se ne accorge, e un sistema che si fida del risultato agisce su un dato sbagliato. Crash, hang e rilevamenti sono invece errori *rilevati* (*DUE, detected unrecoverable errors*): il sistema sa che qualcosa è andato storto e può reagire (riavviare, ripetere il calcolo, passare a uno stato sicuro).

Lo scopo delle tecniche che studiamo è quindi **trasformare le SDC in rilevamenti**. Non correggono l'errore: lo segnalano.

3.4 Protezione hardware e protezione software

Ci si può proteggere in hardware: memorie con codici di correzione (ECC), triplicazione dei circuiti con votazione (TMR), processori che eseguono lo stesso programma in parallelo e confrontano i risultati (*lockstep*). Funziona bene, ma costa: chip speciali, più area, più consumo.

L'alternativa è la **SIHFT** (*Software-Implemented Hardware Fault Tolerance*): si trasforma il programma in modo che si accorga da solo dei guasti, e lo si esegue su hardware normale ed economico. Il prezzo si paga in **tempo di esecuzione** (il programma fa più lavoro) e in **dimensione del codice**.

Le tecniche SIHFT si dividono in due famiglie:

- **protezione del flusso dei dati** (*data-flow*): si duplica il calcolo e si confrontano le due copie. Se un guasto altera una sola copia, il confronto lo rivela;
- **controllo del flusso di esecuzione** (*control-flow checking*, CFC): a ogni basic block si associa una firma e durante l'esecuzione si verifica che i blocchi vengano eseguiti in un ordine permesso dal grafo di controllo del flusso. Serve contro i guasti che fanno saltare il programma nel posto sbagliato.

Le due famiglie sono complementari, e si possono combinare.

3.5 Modelli di guasto

Per valutare una tecnica bisogna decidere quali guasti simulare. Un **modello di guasto** descrive il guasto in modo astratto. I più usati:

- **bit-flip in un registro**: un bit di un registro del processore viene invertito in un istante qualsiasi. Rappresenta i guasti nel banco dei registri e, approssimativamente, quelli nelle unità di calcolo;
- **salto condizionale invertito**: un salto condizionale va dalla parte opposta a quella decisa dalla condizione. Rappresenta un guasto nei flag o nell'unità che gestisce i salti. Il programma resta su archi "permessi" del grafo, ma sbaglia strada;
- **salto a un indirizzo sbagliato**: il program counter viene alterato e il programma continua da un'istruzione qualsiasi;
- **bit-flip in memoria**: un bit di una cella di memoria viene invertito.

Nel progetto usiamo i primi due (capitolo 9).

3.6 Fault injection

Per misurare quanto è efficace una protezione si fa **fault injection**: si esegue il programma molte volte, ogni volta con un guasto artificiale in un punto scelto a caso, e si contano gli esiti. Il guasto si può iniettare simulando l'hardware, con apparecchiature fisiche (fasci di particelle, laser), oppure via software, modificando lo stato del programma mentre è in esecuzione. Noi usiamo l'ultima strada: un debugger (gdb) ferma il programma sotto test e ne modifica i registri.

Siccome i guasti sono scelti a caso, ogni percentuale misurata è una **stima**, e va accompagnata da un **intervallo di confidenza**: con 2000 guasti una percentuale dello 0,1% significa 2 casi, e la vera probabilità potrebbe essere un po' più alta o più bassa. Il capitolo 10 spiega come si calcola l'intervallo.

4. Le tecniche implementate da ASPIS

4.1 EDDI: duplicare le istruzioni

EDDI (*Error Detection by Duplicated Instructions*, Oh, Shirvani e McCluskey, 2002) è la tecnica di base della protezione dei dati. L'idea: ogni istruzione viene eseguita due volte, su due copie separate dei dati, e prima dei punti "pericolosi" si controlla che le due copie coincidano.

In pseudo-C, un'istruzione `a = b + c; x[i] = a;` diventa:

```
a  = b  + c;          // copia originale
a2 = b2 + c2;         // copia duplicata, su variabili separate
if (a != a2 || i != i2) DataCorruption_Handler();   // controllo
x[i]  = a;            // la store originale
x2[i2] = a2;          // e la sua copia, su un array duplicato
```

Ogni variabile ha la sua copia (`b2`, `c2`, `x2`…): locali, globali, e anche la memoria a cui puntano i puntatori, quando è possibile. Un guasto che altera un valore di una sola copia viene scoperto al primo controllo che coinvolge quel valore.

Dove si controlla. Controllare dopo ogni istruzione costerebbe troppo. EDDI controlla nei **punti di sincronizzazione**, cioè dove un valore sbagliato "esce" dal calcolo duplicato e diventa difficile da recuperare:

- prima di una **store**: un valore sbagliato scritto in memoria verrebbe letto da altre parti del programma (si confrontano il valore e l'indirizzo);
- prima di un **salto condizionale**: una condizione sbagliata manderebbe il programma sulla strada sbagliata (si confronta la condizione);
- prima di una **chiamata**: gli argomenti sbagliati verrebbero passati alla funzione chiamata.

Costo. Il programma esegue circa il doppio delle istruzioni, più i confronti e i salti dei controlli: nei nostri programmi EDDI esegue circa 3,7 volte le istruzioni dell'originale (con `-O0`, dove ogni valore passa dallo stack, il costo relativo è più alto che con il codice ottimizzato).

Cosa non vede. Se un guasto colpisce un valore *prima* che venga duplicato (per esempio un dato letto da un file) o colpisce entrambe le copie allo stesso modo, i confronti non lo vedono. Soprattutto, EDDI esegue **un solo salto** per ogni salto del programma: le due copie della condizione vengono confrontate, poi si salta usando la prima. Se è il salto stesso a sbagliare direzione (guasto nei flag o nell'unità di salto), le due copie dei dati seguono insieme la strada sbagliata e restano coerenti tra loro: EDDI non se ne accorge.

Le varianti in ASPIS. Tutte sono implementate dallo stesso file (`passes/EDDI.cpp`) con interruttori diversi:

Opzione	Tecnica	Differenza rispetto a EDDI
`--eddi`	EDDI	duplica tutto, controlla prima di store, salti e chiamate
`--seddi`	sEDDI (*selective EDDI*)	non controlla prima delle store: costa meno, rileva più tardi
`--fdsc`	FDSC (*Full Duplication with Selective Checking*)	duplica tutto ma controlla solo nei basic block che hanno più predecessori
`--reddi`	REDDI	protegge solo le funzioni e le globali annotate con `to_harden` e quello che usano (le funzioni che chiamano, per esempio)
`--multiple-errbb`		un blocco di errore separato per ogni controllo (utile per capire quale controllo è scattato)
`--coarse-grained`	REPAIR	raggruppa le istruzioni duplicate dopo quelle originali, per allontanarle nel tempo

4.2 Il controllo del flusso a firme

Le tecniche di **control-flow checking** proteggono dai guasti che fanno eseguire i blocchi nell'ordine sbagliato. L'idea comune: ogni basic block ha una **firma** (un numero scelto dal compilatore), e il programma tiene in una variabile la **firma di esecuzione**, aggiornata a ogni passaggio da un blocco all'altro. All'ingresso di ogni blocco si controlla che la firma di esecuzione sia quella del blocco: se il programma è arrivato lì da un punto non previsto, la firma non torna e viene chiamato `SigMismatch_Handler`.

Gli errori di flusso si possono classificare così:

1. salto verso un blocco che **non** è un successore del blocco corrente (arco inesistente nel grafo);
2. salto nel **mezzo** di un blocco;
3. salto verso il successore **sbagliato** tra quelli ammessi: è il caso del salto condizionale invertito, dove l'arco esiste ma non è quello scelto dalla condizione.

Le tecniche differiscono in quali di questi errori rilevano.

4.3 CFCSS

CFCSS (*Control-Flow Checking by Software Signatures*, Oh, Shirvani e McCluskey, 2002). Ogni blocco `i` ha una firma statica `s_i`, diversa per ogni blocco. Per ogni blocco con un solo predecessore `p` il compilatore calcola la differenza `d_i = s_p XOR s_i`. La firma di esecuzione `G` viene aggiornata all'ingresso di ogni blocco e confrontata:

```
G = G ^ d_i;                      // all'ingresso del blocco i
if (G != s_i) SigMismatch_Handler();
```

Esempio con firme di 4 bit: il blocco 1 ha due successori, 2 e 3.

```
s1 = 0001     s2 = 0010     s3 = 0100
d2 = s1 XOR s2 = 0011
d3 = s1 XOR s3 = 0101

esecuzione 1 -> 2:   G = 0001;  all'ingresso di 2: G = 0001 XOR 0011 = 0010 = s2   OK
```

Se un guasto fa saltare dal blocco 1 a un blocco 4 che non è suo successore (per esempio un blocco il cui predecessore è 2, con `d4 = s2 XOR s4`), all'ingresso di 4 si ottiene `G = s1 XOR s2 XOR s4`, diverso da `s4`: **rilevato** (errore di tipo 1).

Ma se il salto condizionale del blocco 1 viene **invertito** e si va in 3 invece che in 2:

```
esecuzione 1 -> 3 (sbagliata):  G = 0001 XOR 0101 = 0100 = s3   nessun errore!
```

L'arco 1 -> 3 esiste, quindi CFCSS lo considera lecito: **non rilevato** (errore di tipo 3). CFCSS controlla che il programma segua *un* arco del grafo, non che segua *quello giusto*.

Per i blocchi con più predecessori serve un accorgimento, la firma di aggiustamento `D`: il blocco predecessore imposta `D` prima di saltare, e il blocco di destinazione calcola `G = G XOR d_i XOR D`, in modo che il risultato sia `s_i` da qualunque predecessore lecito si arrivi.

4.4 RASM

RASM (*Random Additive Signature Monitoring*, Vankeirsbilck e altri, 2017) usa firme casuali e operazioni di somma e sottrazione, e soprattutto aggiorna la firma **prima di uscire** da un blocco in base al successore scelto. Ogni blocco `B` ha due numeri casuali: la firma `sig(B)` e un correttivo `sub(B)`. Nel codice di ASPIS (`passes/RASM.cpp`):

```
// all'ingresso del blocco B
S = S - sub(B);
if (S != sig(B)) SigMismatch_Handler();
...  // corpo del blocco
// alla fine di B, prima del salto condizionale "if (c) goto T; else goto F;"
adj_T = sig(B) - (sig(T) + sub(T));      // costanti calcolate dal compilatore
adj_F = sig(B) - (sig(F) + sub(F));
S = S - (c ? adj_T : adj_F);              // una select, non un salto
if (c) goto T; else goto F;
```

Dopo l'aggiornamento `S` vale `sig(T) + sub(T)` se `c` è vero, `sig(F) + sub(F)` se è falso: proprio il valore che il blocco di destinazione si aspetta. Esempio numerico:

```
sig(B) = 100;   sig(T) = 37, sub(T) = 5;   sig(F) = 58, sub(F) = 9
adj_T = 100 - 42 = 58          adj_F = 100 - 67 = 33

c vero, si va in T (corretto):   S = 100 - 58 = 42;  in T: 42 - 5 = 37 = sig(T)   OK
c vero, ma il salto va in F:     S = 42;              in F: 42 - 9 = 33 != 58     RILEVATO
```

Il punto chiave è che la firma viene aggiornata usando **la stessa condizione** del salto, ma con un'istruzione diversa (una `select`) eseguita **prima** del salto. Se un guasto fa andare il salto dalla parte opposta alla condizione, la firma "racconta" un percorso diverso da quello effettivo e il controllo all'ingresso se ne accorge. È per questo che RASM rileva i salti invertiti e CFCSS no.

RASM non vede invece un guasto che altera la condizione `c` stessa prima che venga usata: in quel caso select e salto sono d'accordo tra loro. Ma quello è un guasto nei dati, ed è il lavoro di EDDI. Per questo le due famiglie si combinano bene.

In ASPIS ci sono anche due varianti di RASM che non usiamo nella campagna: **inter-RASM**, che estende il controllo attraverso chiamate e ritorni di funzione, e **RACFED** (Vankeirsbilck e altri, 2018), che aggiorna la firma anche dopo ogni istruzione del blocco, per rilevare i salti nel mezzo di un blocco.

4.5 Combinare dati e flusso

ASPIS permette di applicare una tecnica di protezione dei dati e una di controllo del flusso allo stesso programma (`--eddi --rasm`). La protezione dei dati viene applicata per prima, quella del flusso dopo, sul codice già dupli-

cato: quindi anche i blocchi creati da EDDI per i suoi controlli ricevono firme e controlli di RASM. Proprio in questa interazione si nascondeva il bug di RASM (capitolo 8).

Parte II. Il lavoro fatto

5. ASPIS in pratica

5.1 Il repository di ASPIS

ASPIS (*Automatic Software-based Protection and Integrity Suite*) è sviluppato dal gruppo HEAPLab del Politecnico (articolo di riferimento: Baroffio, Reghenzani, Fornaciari, ACM TODAES 2024) ed è su GitHub in `HEAPLab/ASPIS`. Abbiamo lavorato sul commit `b66a090`, l'ultimo del branch principale quando è iniziato il progetto. Le parti che ci interessano:

Percorso	Contenuto
`aspis.sh`	lo script che compila un programma applicando le protezioni
`passes/EDDI.cpp`	EDDI e le sue varianti (sEDDI, FDSC, REDDI): circa 2400 righe
`passes/FuncRetToRef.cpp`	trasforma i valori di ritorno in parametri puntatore
`passes/DuplicateGlobals.cpp`	duplica le variabili globali
`passes/CFCSS.cpp`, `RASM.cpp`, `RACFED.cpp`	le tecniche di controllo del flusso (inter-RASM è RASM compilato con un'opzione)
`passes/ASPIS.h`	le dichiarazioni delle classi dei pass
`libs/TypeDeductionAnalysis/`	la libreria che deduce i tipi puntati (un *submodule* git)
`testing/`	il sistema di test: `test.py`, `config/tests.toml`, `tests/` con i programmi

5.2 Compilare ASPIS

Serve LLVM 21.1.8 (scaricato già compilato), CMake e Ninja:

```
git clone --recursive https://github.com/HEAPLab/ASPIS
cd ASPIS && git checkout b66a090
cmake -B build -G Ninja -DLLVM_DIR=<llvm>/lib/cmake/llvm
cmake --build build
```

Il risultato sono le librerie dei pass in `build/passes/`: `libEDDI.so`, `libSEDDI.so`, `libFDSC.so`, `libREDDI.so`, `libCFCSS.so`, `libRASM.so`, `libINTER_RASM.so`, `libRACFED.so`. La compilazione richiede circa un minuto.

5.3 Usare ASPIS

```
./aspis.sh --llvm-bin <llvm>/bin --eddi --rasm programma.c -o programma
```

Le opzioni principali: una tecnica dei dati (`--no-dup`, `--eddi`, `--seddi`, `--fdsc`, `--reddi`, con eventuali `--multiple-errbb` e `--coarse-grained`), una tecnica del flusso (`--no-cfc`, `--cfcss`, `--rasm`, `--inter-rasm`, `--racfed`), `--build-dir` per la cartella dei file intermedi e `--no-cleanup` per non cancellarli alla fine. Con `--no-cleanup` nella cartella resta `out.ll`, l'IR finale: è il file da guardare quando qualcosa non funziona.

Il programma deve definire le due funzioni che ASPIS chiama quando un controllo fallisce:

```c
void DataCorruption_Handler(void) {     /* controlli sui dati (EDDI & co.) */
    fprintf(stderr, "FAULT_DETECTED: DataCorruption\n");
    exit(2);
}
void SigMismatch_Handler(void) {        /* controlli sulle firme (CFC) */
    fprintf(stderr, "FAULT_DETECTED: SigMismatch\n");
    exit(3);
}
```

5.4 La pipeline di `aspis.sh`

Questi sono, nell'ordine, i comandi che `aspis.sh` esegue (semplificati):

```
# 1. ogni sorgente diventa IR, senza ottimizzazioni
clang file.c -S -emit-llvm -O0 -Xclang -disable-O0-optnone -o file.ll
# 2. tutti i file IR vengono uniti in un solo modulo
llvm-link *.ll -o out.ll
# 3. preparazione: via le informazioni di debug, switch trasformati in br
opt --passes="strip" out.ll -o out.ll
opt --passes="lower-switch" out.ll -o out.ll
# 4. i valori di ritorno diventano parametri (vedi 5.5)
opt -load-pass-plugin=libEDDI.so --passes="func-ret-to-ref" out.ll -o out.ll
# 5. protezione dei dati: libEDDI.so, libSEDDI.so, libFDSC.so o libREDDI.so
opt -load-pass-plugin=libEDDI.so --passes="eddi-verify" out.ll -o out.ll
# 6. semplificazione del grafo dei blocchi
opt --passes="simplifycfg" out.ll -o out.ll
# 7. controllo del flusso: cfcss-verify, rasm-verify o racfed-verify
opt -load-pass-plugin=libRASM.so --passes="rasm-verify" out.ll -o out.ll
# 8. coerenza delle variabili globali duplicate
opt -load-pass-plugin=libEDDI.so --passes="duplicate-globals" out.ll -o out.ll
# 9. back-end: IR -> eseguibile
clang out.ll -o programma
```

Due osservazioni:

- l'IR viene prodotto con `-O0` e **nessun passo di ottimizzazione** viene eseguito dopo le protezioni. È voluto: un ottimizzatore riconoscerebbe che le due copie calcolano la stessa cosa e le fonderebbe, cancellando la protezione. Il prezzo è un codice lento, con ogni variabile sullo stack;
- la protezione del flusso viene applicata **dopo** quella dei dati, quindi lavora sul codice già duplicato, compresi i blocchi creati da EDDI.

5.5 FuncRetToRef

Una funzione duplicata deve restituire **due** risultati, uno per copia, ma in C (e in LLVM) una funzione restituisce un solo valore. `FuncRetToRef` risolve il problema prima di EDDI: per ogni funzione `int f(int x)` crea una versione `void f_ret(int x, int *ret)` che scrive il risultato attraverso un puntatore, e sostituisce le chiamate. EDDI poi duplica anche il puntatore di ritorno, così le due copie del risultato finiscono in due posti diversi.

5.6 Come EDDI trasforma il codice

Per ogni funzione `f` del programma, EDDI crea una versione `f_dup` che riceve **ogni parametro due volte** (prima le copie duplicate, poi gli originali) e dentro la quale ogni istruzione è eseguita due volte. Mantiene una mappa

"istruzione originale – istruzione copia". Ogni variabile globale `g` ottiene una copia `g_dup`.

Ai punti di sincronizzazione (store, salti condizionali, chiamate) la funzione `addConsistencyChecks` (in `EDDI.cpp`):

1. spezza il basic block prima dell'istruzione da proteggere;
2. inserisce un nuovo blocco chiamato **`VerificationBB`** con i confronti tra le due copie degli operandi (`icmp eq` per gli interi, `fcmp ueq` per i numeri in virgola mobile, confronto dei valori puntati per i puntatori);
3. termina il blocco con un salto: se tutti i confronti sono veri si prosegue con l'istruzione, altrimenti si va in **`ErrBB`**, un blocco che chiama `DataCorruption_Handler`.

Questo è l'esempio del report: `int scale(int v) { return v * g; }`, con `g` globale, compilato con EDDI:

```
define void @scale_ret_dup(i32 %0, ptr %1, i32 %2, ptr %3) {
VerificationBB:
  %4 = alloca i32                       ; v
  %5 = alloca i32                       ; la sua copia
  store i32 %2, ptr %4
  store i32 %0, ptr %5
  %6 = load i32, ptr %4
  %7 = load i32, ptr %5
  %8 = load i32, ptr @g
  %9 = load i32, ptr @g_dup             ; la copia della globale
  %10 = mul nsw i32 %6, %8
  %11 = mul nsw i32 %7, %9              ; il calcolo è fatto due volte
  %12 = icmp eq i32 %10, %11            ; controllo di consistenza
  br i1 %12, label %13, label %ErrBB
13:
  store volatile i32 %10, ptr %3        ; "restituisce" le due copie
  store volatile i32 %11, ptr %1
  ret void
ErrBB:
  call void @DataCorruption_Handler()
  unreachable
}
```

Si riconoscono tutti gli ingredienti: il nome `scale_ret_dup` (prima `FuncRetToRef` ha creato `scale_ret`, poi EDDI `scale_ret_dup`), i quattro parametri (`%0` e `%2` sono le due copie di `v`, `%1` e `%3` i due puntatori di ritorno), le operazioni raddoppiate, il confronto prima delle store che fanno uscire il risultato, il blocco `ErrBB`. Il primo blocco si chiama `VerificationBB` perché EDDI lo ha creato spezzando il blocco d'ingresso.

Le chiamate. Se la funzione chiamata ha una versione `_dup` (perché è nel programma ed è stata duplicata), EDDI chiama quella, passando entrambe le copie degli argomenti. Se invece la funzione è esterna (`printf`, `memcpy`, `sin` della libreria C), non può essere duplicata: EDDI la chiama una volta sola con gli argomenti originali. Ma una funzione esterna può **scrivere** attraverso i puntatori che riceve (pensate a `sprintf(buf, ...)` o `scanf`), aggiornando solo la copia originale dei dati. Per questo, dopo la chiamata, EDDI **risincronizza** la memoria: copia i valori puntati dagli argomenti dalla versione originale a quella duplicata. È la funzione `fixFuncValsPassedByReference`, dove sta il bug di EDDI che abbiamo trovato (capitolo 8). (`memcpy`, `memset` e simili invece non sono chiamate esterne: nell'IR sono *intrinsic* di LLVM, che EDDI duplica come le altre istruzioni.)

I puntatori. Le due copie di un puntatore possono legittimamente avere valori diversi: se `p` punta a `x`, la sua copia punta a `x_dup`. Quindi confrontare gli indirizzi non ha senso: EDDI confronta i **valori puntati** (`*p` con `*p_dup`), e se sono a loro volta puntatori scende ancora, fino a un valore non puntatore. Per sapere di che tipo è il valore pun-

tato (con i puntatori opachi, vedi 2.6) usa i tipi dedotti da TypeDeductionAnalysis (funzioni `comparePtrs` e `compareValues`).

5.7 Le altre parti

- **DuplicateGlobals** chiude il lavoro sulle variabili globali: le globali scritte da funzioni non duplicate vengono mantenute coerenti con le loro copie;
- le **annotazioni** permettono di scegliere cosa proteggere: `__attribute__((annotate("exclude")))` lascia una funzione o una globale senza protezione, `to_duplicate` duplica le chiamate a una funzione invece del suo corpo, `to_harden` indica cosa deve proteggere REDDI. Le nostre applicazioni non usano annotazioni: vogliamo proteggere tutto;
- EDDI, a fine compilazione, stampa `Comparison Counter: N`, il numero di controlli inseriti; lo usiamo come misura della quantità di controlli.

5.8 Come si vede RASM nel codice

RASM tiene la firma di esecuzione in una variabile sullo stack (`%2` nell'esempio sotto), letta e scritta con accessi `volatile` perché il compilatore non li elimini. Per ogni blocco `B` crea un blocco `RASM_Verification_BB`, lo mette davanti a `B` e ci fa arrivare tutti i salti che andavano in `B`: lì sottrae il correttivo e controlla la firma. Alla fine di ogni blocco aggiunge l'aggiornamento con la `select` (capitolo 4.4).

Questo è un estratto di una piccola funzione compilata con `--eddi --rasm` (`void clamp(int v) { if (v > 10) g = 10; else g = v; }`):

```
14:
  %15 = select i1 %8, i32 -3, i32 -5          ; correttivo scelto con la condizione
  %16 = load volatile i32, ptr %2             ; firma di esecuzione
  %17 = sub i32 %16, %15
  store volatile i32 %17, ptr %2              ; firma aggiornata
  br i1 %8, label %RASM_Verification_BB2, label %RASM_Verification_BB3

RASM_Verification_BB2:                        ; ingresso del ramo "vero"
  %18 = load volatile i32, ptr %2
  %19 = sub i32 %18, 1                        ; S = S - sub(B)
  store volatile i32 %19, ptr %2
  %20 = icmp eq i32 %19, 24                   ; S == sig(B) ?
  br i1 %20, label %VerificationBB2, label %ErrBB1

VerificationBB2:                              ; blocco creato da EDDI per la store
  store i32 10, ptr @g
  store i32 10, ptr @g_dup
  ...
```

Si vede bene l'intreccio delle due protezioni: il salto condizionale porta prima al controllo di RASM, poi al blocco `VerificationBB2` creato da EDDI. Nel capitolo 8 vedremo cosa succedeva prima della correzione del bug di RASM.

6. Le quattro applicazioni

6.1 Perché queste e con quali regole

La proposta lascia libera la scelta del codice da proteggere. Abbiamo scritto quattro programmi piccoli ma realistici, scelti per coprire tipi di calcolo diversi e tipici dei sistemi embedded, dove si usa la SIHFT: crittografia (operazioni su byte e tabelle), elaborazione del segnale (virgola mobile a doppia precisione), compressione (strutture dati e salti che dipendono dai dati), controllo automatico (virgola mobile a singola precisione, stato globale).

Tutti seguono le stesse regole, ciascuna con un motivo preciso:

Regola	Perché
deterministici: nessun input esterno, nessun numero casuale vero	ogni esecuzione senza guasti deve stampare esattamente lo stesso output, altrimenti non si potrebbe confrontare l'output con quello di riferimento
autoverificanti: confrontano i risultati con valori attesi e stampano `SUCCESS` o `FAIL` (codice d'uscita 0 o 1)	un risultato sbagliato si vede sia nell'output sia nel codice d'uscita
definiscono i due **gestori d'errore**, che stampano `FAULT_DETECTED: ...` su stderr ed escono con codice 2 o 3	un rilevamento si riconosce subito, sia nei test sia nella fault injection
sono validi sia in **C sia in C++**	il sistema di test di ASPIS compila il programma di riferimento con `clang++`
sono **brevi**: meno di un milione di istruzioni eseguite	migliaia di esperimenti di fault injection devono durare minuti, non giorni
usano solo la **libreria standard** e sono rilasciati con **licenza MIT**	possono entrare nel repository di ASPIS

Ogni file inizia con un commento che spiega cosa fa il programma e quali caratteristiche del C mette alla prova.

6.2 `aes128.c`: cifratura AES-128

329 righe, 645.656 istruzioni eseguite. Implementa l'algoritmo di cifratura a blocchi AES con chiave di 128 bit (standard FIPS-197):

- calcola all'avvio le tabelle `sbox` e `inv_sbox` (la sostituzione di byte di AES) con l'aritmetica nel campo finito GF(2^8), e le salva in array globali **non costanti**, così ASPIS deve duplicarli;
- esegue il *known-answer test* dell'appendice C.1 dello standard: cifra un blocco noto con una chiave nota e confronta il risultato con quello pubblicato (`69c4e0d86a7b0430d8cdb78070b4c55a`), poi lo decifra;
- cifra e decifra in modalità CBC un messaggio di 64 byte e verifica che il testo decifrato sia uguale all'originale; stampa anche un hash (FNV-1a) del testo cifrato.

Stampa le tabelle e i risultati intermedi e finisce con `SUCCESS` se tutti i confronti tornano. Nella prima versione la funzione `print_hex` riceveva un'etichetta come parametro e la stampava con `printf("%s", label)`: era proprio il caso che faceva andare in crash EDDI (capitolo 8). Ora l'etichetta viene stampata da chi chiama `print_hex`, con una stringa letterale.

6.3 `fft.c`: trasformata di Fourier

186 righe, 367.747 istruzioni. Calcola la FFT radix-2 (algoritmo di Cooley-Tukey iterativo) di un segnale di 256 campioni complessi in doppia precisione:

- il segnale è la somma di tre sinusoidi (frequenze 5, 12 e 40) più un rumore deterministico;
- trova le tre frequenze dominanti e controlla che siano 5, 12 e 40;
- verifica il **teorema di Parseval** (l'energia del segnale nel tempo è uguale a quella in frequenza, divisa per N);
- calcola la trasformata inversa e controlla che ricostruisca il segnale con un errore inferiore a 10^-9.

Usa la libreria matematica (`sin` , `cos` , `sqrt`), e infatti va compilato con `-lm` .

6.4 `huffman.c` : compressione di Huffman

225 righe, 170.962 istruzioni. Comprime e decomprime un testo fisso con la codifica di Huffman:

- conta le frequenze dei caratteri;
- costruisce l'albero di Huffman con un **min-heap** binario (con un criterio deterministico per i casi di parità);
- assegna i codici con una visita ricorsiva dell'albero;
- scrive il testo compresso in un buffer di bit globale e poi lo decodifica percorrendo l'albero;
- controlla che il testo decodificato sia uguale all'originale e stampa statistiche e un checksum (Adler-32) dei dati compressi.

L'albero è un array di struct con indici interi al posto dei puntatori, e i cicli del heap hanno uscite che dipendono dai dati: tanti salti condizionali "interessanti" per il controllo del flusso.

6.5 `pid_control.c` : controllore PID

159 righe, 477.379 istruzioni. Simula per 2000 passi di 10 ms un motore in corrente continua (modello del primo ordine) controllato da un **PID**:

- il riferimento di velocità cambia a gradini (10, 25, 5, 15 rad/s);
- tra il passo 1200 e il 1300 si applica un disturbo di coppia;
- il sensore di velocità è quantizzato come un encoder;
- il PID ha la derivata filtrata, la saturazione dell'uscita a ±24 V e un meccanismo *anti-windup* (l'integrale viene aggiornato solo se aiuta a uscire dalla saturazione).

Stampa ogni 250 passi lo stato del sistema e alla fine l'errore integrale assoluto (IAE), la sovraelongazione e i passi in saturazione; il controllo finale verifica che la velocità abbia raggiunto il riferimento. Usa `float` (singola precisione), tipico dei microcontrollori, e tiene lo stato del controllore in una struct globale aggiornata tramite puntatore.

7. I test di compatibilità

7.1 Il sistema di test di ASPIS

ASPIS ha un sistema di test basato su **pytest** (cartella `testing/`). I programmi di test sono elencati in `testing/config/tests.toml`, uno per voce:

```
[[tests]]
test_name = "c_fft"
source_file = "c/applications/fft.c"     # relativo a testing/tests/
add_compiler_flags = "-lm"               # opzioni in più per aspis.sh
# black_list = ["--inter-rasm"]          # tecniche da non provare
```

Il file `testing/test.py` prende ogni test e lo compila con **55 combinazioni**: le 11 opzioni dei dati (`--no-dup`, `--eddi`, `--eddi --multiple-errbb`, `--eddi --coarse-grained`, `--eddi --multiple-errbb --coarse-grained`, le stesse quattro per `--reddi`, `--seddi`, `--fdsc`) per le 5 del flusso (`--no-cfc`, `--cfcss`, `--rasm`, `--racfed`, `--inter-rasm`). Per ogni combinazione:

1. compila il programma con `clang++` senza ASPIS (una volta sola) e lo esegue: è l'output di riferimento;
2. compila il programma con `aspis.sh` e le opzioni della combinazione;
3. lo esegue; il test **passa** se l'esecuzione termina con codice 0 e l'output è uguale a quello di riferimento.

Ogni combinazione ha un limite di 60 secondi. Le combinazioni nella `black_list` di un test vengono saltate (*skipped*).

Si lancia così, dalla cartella `testing/` di ASPIS (dopo aver scritto il percorso di LLVM in `testing/config/llvm.toml`):

```
pytest test.py -v > harness.log
```

Ogni riga del log è una combinazione, per esempio `test.py::test_aspis[c_fft---rasm---eddi] PASSED`: nome del test, tecnica del flusso, tecnica dei dati, esito.

7.2 Cosa abbiamo fatto

Abbiamo aggiunto le quattro applicazioni in `testing/tests/c/applications/` e le loro voci in fondo a `tests.toml` (sono nel file `aspis-integration/flc_tests.toml`), poi abbiamo eseguito l'intera suite tre volte (log in `results/compatibility/`):

- con la **prima versione** delle applicazioni, sul branch principale di ASPIS (`b66a090`): `harness_master_first_version.log.gz`;
- con la **versione finale** delle applicazioni (quella che evita il bug di EDDI), sul branch principale: `harness_master.log.gz`;
- con la versione finale e le **nostre due patch** (la correzione di RASM e le voci dei test): `harness_fixed.log.gz`.

Ognuna delle esecuzioni compila e prova 52 test × 55 combinazioni = 2860 programmi e dura circa 9 minuti. Lo script `fault_injection/analyze.py` legge i log e produce le tabelle del report.

7.3 I risultati

Le nostre applicazioni (4 applicazioni × 5 tecniche del flusso per ogni riga):

Protezione dei dati	prima versione, `master`	versione finale, `master`	versione finale, con le patch
nessuna (`--no-dup`)	20 / 20	20 / 20	20 / 20
EDDI, 4 varianti	0 / 80	80 / 80	80 / 80
sEDDI	0 / 20	20 / 20	20 / 20
FDSC	0 / 20	20 / 20	20 / 20
REDDI, 4 varianti	80 / 80	80 / 80	80 / 80
totale	**100 / 220**	**220 / 220**	**220 / 220**

Come si legge:

- **le tecniche del flusso da sole funzionano** su tutte le applicazioni (riga `--no-dup`);
- **REDDI passa**, ma per un motivo banale: REDDI protegge solo ciò che è annotato con `to_harden` (e quello che usa), e le nostre applicazioni non hanno annotazioni, quindi REDDI non protegge nulla;
- con la prima versione **EDDI, sEDDI e FDSC andavano in crash su tutte e quattro le applicazioni** (*segmentation fault*), con qualunque tecnica del flusso: è il bug di EDDI del capitolo 8;
- con la versione finale, che evita quel bug, passano tutte le 220 combinazioni, sia su `master` sia con le nostre patch.

I 48 test che ASPIS aveva già:

Esito	`master`	con le patch
passati	2453	2453
falliti	33	33
saltati	154	154

I risultati sono **identici**: la correzione di RASM non rompe nulla (nessuna *regressione*). I 154 saltati sono le combinazioni nelle `black_list`. I 33 falliti sono tutte le combinazioni non escluse di un solo test, `cpp_complex`, e falliscono anche su `master` già nel primo passo: il programma di riferimento non compila, perché il sorgente usa `std::powf`, che la libreria C++ della nostra macchina (libstdc++ 13) non dichiara. È un problema del test e dell'ambiente, non di ASPIS né delle nostre modifiche.

8. I due bug trovati in ASPIS

Il progetto ha trovato due bug di ASPIS:

- un bug di **EDDI**, trovato dai test di compatibilità, che faceva andare in crash la prima versione delle nostre applicazioni. Non lo abbiamo corretto dentro ASPIS: lo abbiamo isolato in un programma di cinque righe, descritto in un *bug report* e aggirato nelle applicazioni;
- un bug di **RASM**, trovato dalla fault injection, che abbiamo corretto cambiando una riga di codice.

8.1 Come si cerca un bug in un pass

Quando un programma protetto da ASPIS non funziona, il metodo è sempre lo stesso:

1. **riprodurre**: trovare la combinazione di opzioni che fallisce (il log del sistema di test la dice) e ripetere la compilazione a mano con `aspis.sh ... --no-cleanup`, così restano i file intermedi;
2. **capire dove**: se il programma va in crash, eseguirlo con `gdb` (`run`, poi `bt` per lo stack delle chiamate e `x/10i $pc` per le istruzioni vicine); poi guardare l'IR in `out.ll` intorno al punto incriminato;
3. **ridurre**: scrivere un programma minimo, di poche righe, che mostra lo stesso problema (un *riproduttore*). Serve a capire la causa e a comunicarla a chi mantiene ASPIS;
4. **leggere il codice del pass** che produce l'IR sbagliato e capire perché;
5. **decidere**: correggere ASPIS (e verificare che tutti i suoi test diano gli stessi risultati di prima) oppure, se la correzione è complessa, segnalare il bug ed evitarlo.

8.2 Il bug di EDDI: la copia dopo le chiamate di libreria

Sintomo. Con la prima versione delle applicazioni, EDDI, sEDDI e FDSC le facevano andare in crash **tutte e quattro** (*segmentation fault*). Con `gdb` si vede che il crash è sempre una scrittura di 8 byte (`mov %rcx,(%rax)`) subito dopo una chiamata a `printf`, e che l'indirizzo scritto è una **stringa letterale**, che sta in memoria di sola lettura (la sezione `.rodata`):

Applicazione	Stringa sovrascritta	Codice C della prima versione
`aes128`	`"kat ct: "`	`print_hex("kat ct: ", ...)`, che faceva `printf("%s", label)`
`fft`	`"OK"`	`printf("ifft reconstruction: %s\n", max_err < 1e-9 ? "OK" : "MISMATCH")`
`huffman`	`"OK"`	`printf("roundtrip: %s\n", ok ? "OK" : "MISMATCH")`
`pid_control`	`"SUCCESS"`	`printf("%s\n", ok ? "SUCCESS" : "FAIL")`

Il riproduttore `aspis-integration/reproducers/string_literal.c` è di cinque righe:

```
static void show(const char *label) { printf("%s\n", label); }
int main(void) { show("hello"); return 0; }
```

Compilato con `--eddi` sul branch principale di ASPIS va in crash invece di stampare `hello`.

Causa. Ricordate la risincronizzazione dopo le chiamate esterne (5.6): dopo una chiamata a una funzione che non ha una versione `_dup` (come `printf`), EDDI copia i valori puntati dagli argomenti dalla versione originale a quella duplicata, perché la funzione chiamata potrebbe averli modificati. Il codice (`fixFuncValsPassedByRefe-`

`rence` in `EDDI.cpp`) fa, per ogni argomento che sia il risultato di un'istruzione e abbia una copia, una lettura e una scrittura di un valore **della dimensione di un puntatore** (8 byte):

```
// codice di ASPIS b66a090 (semplificato)
Instruction *TmpLoad  = B.CreateLoad(Original->getType(), Original);  // legge 8 byte
Instruction *TmpStore = B.CreateStore(TmpLoad, Copy);                 // li scrive nella copia
```

Quando l'argomento è una stringa letterale, la "copia" del puntatore punta **alla stessa stringa**: nel caso di `show("hello")` il parametro `label` e la sua copia valgono entrambi l'indirizzo di `"hello"`; nel caso di `ok ? "SUCCESS" : "FAIL"` l'istruzione `select` (il `? :`) e la sua copia scelgono la stessa costante. La scrittura finisce quindi nella memoria di sola lettura, e il sistema operativo uccide il programma. A parte questo caso, copiare sempre 8 byte è sbagliato anche in generale: per un `int` (4 byte) si scrive oltre la fine della copia, per un buffer di 100 byte se ne copiano solo 8.

Perché `printf("SUCCESS\n")` invece funziona? Perché lì la stringa è passata direttamente come **costante**, non come risultato di un'istruzione, e il codice di EDDI salta le costanti.

Cosa abbiamo fatto. Correggere questa funzione richiede di riscriverla (copiare la dimensione giusta dell'oggetto, saltare la memoria di sola lettura e gli argomenti che la funzione non può modificare): un intervento complesso dentro EDDI. Abbiamo scelto di:

1. **aggirarlo** nelle applicazioni: ora stampano solo stringhe letterali dirette (`if (ok) printf("SUCCESS\n"); else printf("FAIL\n");`), con lo stesso identico output. Un commento in cima a ogni file spiega il motivo. Con questa modifica tutte le 220 combinazioni passano sul branch principale;
2. **segnalarlo**: `aspis-integration/bug_report.md` descrive il bug, il riproduttore, la causa e una possibile correzione, pronto per essere aperto come *issue* sul repository di ASPIS.

La correzione completa esiste (era nella seconda versione del progetto, capitolo 15), ma non fa parte del progetto consegnato.

8.3 Il bug di RASM: i blocchi di EDDI non venivano controllati

Questo bug **non fa fallire nessun programma**: i test di compatibilità non potevano trovarlo. L'ha trovato la fault injection.

Sintomo. Ci si aspetta che EDDI+RASM protegga almeno quanto RASM da solo, visto che aggiunge altri controlli. Invece, contro i salti invertiti, EDDI+RASM lasciava il **9,70%** di SDC, contro lo **0,70%** di RASM da solo (capitolo 11).

Come l'abbiamo trovato. Il file dei risultati grezzi (`results/fault_injection/before_fix/raw.csv.gz`) ha una riga per esperimento, con l'indirizzo dell'istruzione colpita (`pc_off`, la distanza dall'inizio del codice). Prendendo gli esperimenti finiti in SDC e disassemblando il programma a quegli indirizzi (con `objdump -d`), si vede che **tutte** le 194 SDC sono avvenute su salti condizionali verso blocchi che cominciano con la stessa istruzione: `mov DWORD PTR [rbp-0x4], <costante>`, cioè la scrittura di una costante nella variabile della firma di RASM. In altre parole, in quei blocchi RASM non controllava la firma: la reimpostava. In tutti i casi entrambi i successori del salto erano blocchi così. Lo script `fault_injection/sdc_sites.py` fa esattamente questa analisi:

```
$ python3 fault_injection/sdc_sites.py results/fault_injection/before_fix/raw.csv.gz \
      <cartella di lavoro della campagna> huffman EDDI+RASM
huffman EDDI+RASM branch: 64 SDC at 16 instructions
    18 x  bdc3  fn_4_ret_dup         jne    bdca <fn_4_ret_dup+0x19a>
            not taken -> jmp    bfe7 <fn_4_ret_dup+0x3b7>
            taken     -> mov    DWORD PTR [rbp-0x4],0x4d8
    ...
```

(il successore "non preso" è un `jmp` verso `bfe7`, che comincia anch'esso con `mov DWORD PTR [rbp-0x4],0x4e8`).

Causa. In `RASM.cpp`, all'ingresso di ogni blocco, c'è questa scelta:

```
// codice originale
if (isa<LandingPadInst>(BB.getFirstNonPHI()) ||
    BB.getName().contains_insensitive("verification")) {
  // NON controlla la firma: la reimposta al valore atteso del blocco
  BChecker.CreateStore(ConstantInt::get(IntType, randomNumberBB), &RuntimeSig, true);
} else {
  // crea il blocco RASM_Verification_BB che controlla la firma
}
```

L'eccezione serviva per i blocchi creati da RASM stesso (che si chiamano `RASM_Verification_BB...`), dove la firma non va controllata due volte. Ma il test sul nome (`contains_insensitive("verification")`, cioè "il nome contiene *verification*, maiuscole o minuscole") è troppo largo: vale anche per i blocchi `VerificationBB` creati da EDDI. E con EDDI questi blocchi sono dappertutto, perché EDDI ne crea uno prima di ogni store, salto e chiamata: sono le destinazioni di quasi tutti i salti.

Ecco la stessa funzione del capitolo 5.8 (`clamp`) compilata **prima** della correzione:

```
  br i1 %8, label %VerificationBB2, label %VerificationBB3   ; salto condizionale

VerificationBB2:                    ; nessun controllo di RASM davanti
  store volatile i32 24, ptr %2     ; firma REIMPOSTATA al valore atteso, senza controllo
  store i32 10, ptr @g
  ...
VerificationBB3:
  store volatile i32 26, ptr %2     ; idem
  ...
```

Se il salto viene invertito, il programma entra nel blocco sbagliato, ma la prima cosa che fa è sovrascrivere la firma con il valore "giusto" per quel blocco: l'errore viene cancellato prima di poter essere controllato. Con la correzione (capitolo 5.8) il salto va prima in `RASM_Verification_BB2` o `RASM_Verification_BB3`, che controllano la firma.

Correzione (patch 0001). Una riga: la condizione diventa `BB.getName().starts_with("RASM_")`, così l'eccezione vale solo per i blocchi creati da RASM stesso (e per i *landing pad*, i blocchi di gestione delle eccezioni del C++). Lo stesso errore c'era, identico, in `RACFED.cpp` (con `starts_with("RACFED_")`), e la patch corregge anche quello.

Verifica. Abbiamo rifatto gli esperimenti di EDDI+RASM con la correzione: nessuna SDC sui 2000 salti invertiti (prima erano 194), e i controlli di RASM ne rilevano il 67,1% invece del 36,4% (il resto lo rilevano i controlli di EDDI). La tabella completa è nel capitolo 11. Sui 48 test già presenti in ASPIS la correzione non cambia nessun risultato (capitolo 7).

9. Iniettare i guasti con gdb e valgrind

Non abbiamo scritto un iniettore: usiamo due strumenti che si trovano su qualunque Linux (`sudo apt install gdb valgrind`), guidati da due brevi script Python.

- **valgrind** esegue il programma su un processore simulato e, con lo strumento `callgrind`, conta quante volte viene eseguita ogni istruzione. Lo usiamo una volta per programma, nella esecuzione senza guasti (la *golden run*).
- **gdb**, il debugger, sa fermare un programma su qualunque istruzione, leggere e cambiare i registri e farlo ripartire: è esattamente quello che serve per iniettare un guasto. Lo usiamo in ogni esperimento.

Lo script `fault_injection/gdb_inject.py` (72 righe, di cui 45 di codice) fa un esperimento dentro gdb; `fault_injection/campaign.py` prepara gli esperimenti, li lancia e ne classifica l'esito (capitolo 10).

9.1 La golden run

Per ogni programma, `campaign.py`:

1. lo esegue normalmente cinque volte: l'output è quello di riferimento (il *golden output*) e il tempo più breve serve per il limite degli *hang* (9.4);
2. lo esegue una volta sotto valgrind:

   ```
   valgrind --tool=callgrind --dump-instr=yes --dump-line=no \
       --compress-pos=no --compress-strings=no --callgrind-out-file=callgrind.out ./prog
   ```

   Il file `callgrind.out` contiene, per ogni istruzione eseguita, il suo indirizzo e il numero di esecuzioni, raggruppati per file eseguibile (`ob=`): teniamo solo le istruzioni del programma, non quelle della libreria C e del caricatore dinamico;
3. con `objdump -d` segna quali di quelle istruzioni sono salti condizionali (`je`, `jne`, `jl`, ..., tutti i `j...` tranne `jmp`);
4. scrive il profilo `profile.csv`. Le prime righe di quello di `aes128` senza protezione:

   ```
   address,executions,jcc
   1080,1,0
   1084,1,0
   1086,1,0
   ...
   10c1,1,1          # 10c1: je 10d8 <deregister_tm_clones+0x28>, un salto condizionale
   ```

   In tutto 1289 istruzioni diverse e 645.656 esecuzioni: sono le istruzioni eseguite nel codice del programma, la misura del costo del capitolo 11.1. Anche il codice di avvio che il compilatore aggiunge a ogni programma (`_start`, `deregister_tm_clones`) fa parte dell'eseguibile.

Gli indirizzi sono quelli che stampa `objdump`, cioè relativi all'inizio del file: il programma è *position independent* e a ogni esecuzione viene caricato a un indirizzo base diverso, a meno di disattivare la randomizzazione degli indirizzi (ASLR), che gdb disattiva da solo.

9.2 Scegliere l'istante

Un guasto transitorio può arrivare in qualunque momento, quindi vogliamo un istante a caso dell'esecuzione, cioè **un'esecuzione a caso di un'istruzione qualsiasi**. Dal profilo `campaign.py` estrae:

1. un'istruzione, con probabilità proporzionale al numero delle sue esecuzioni (un'istruzione eseguita 1000 volte ha 1000 volte più probabilità di una eseguita una volta);
2. una delle sue esecuzioni, la *k*-esima, con *k* uniforme tra 1 e il numero di esecuzioni.

Così ogni esecuzione di ogni istruzione ha la stessa probabilità di essere scelta. Per i salti invertiti si sceglie solo tra le esecuzioni dei salti condizionali. Per i bit-flip si estraggono anche il registro (uno dei 16 registri generali, da `rax` a `r15`) e il bit (da 0 a 63). I numeri casuali vengono da un generatore inizializzato con un seme ricavato dai nomi di applicazione, configurazione e modello: rifacendo la campagna si ottengono gli stessi esperimenti.

9.3 Un esperimento con gdb

`campaign.py` lancia gdb così, passando l'esperimento nelle variabili d'ambiente (`FI_ADDR`, `FI_K`, `FI_MODEL`, `FI_REG`, `FI_BIT`, …):

```
gdb -batch -nx -iex "set debuginfod enabled off" -x gdb_inject.py --args ./prog
```

e lo script fa, con l'API Python di gdb, quello che si farebbe a mano. Ecco la stessa cosa in una sessione interattiva, per l'esperimento "inverti il bit 11 di `r11` alla 1808ª esecuzione dell'istruzione `0x12ea`" (in `main` di `pid_control` con EDDI, dove viene eseguita 2000 volte):

```
$ gdb ./prog
(gdb) print &main                       # indirizzo di main nel file: 0x1200
(gdb) starti > out.txt 2> err.txt       # avvia il programma e fermalo subito
(gdb) print &main                       # ora vale 0x555555555200: la base è 0x555555554000
(gdb) break *(0x555555554000 + 0x12ea)
(gdb) ignore 1 1807                     # ignora i primi 1807 passaggi
(gdb) continue                          # si ferma prima della 1808ª esecuzione
(gdb) set $r11 = $r11 ^ (1 << 11)       # il guasto
(gdb) delete
(gdb) continue                          # il programma prosegue con il guasto
```

I passi dello script:

1. **avvio**: `starti` avvia il programma (con l'output rediretto su due file) e lo ferma alla prima istruzione; la differenza tra l'indirizzo di `main` prima e dopo l'avvio è la base a cui è stato caricato;
2. **fermata**: un breakpoint sull'istruzione scelta, con *ignore count* *k*-1: gdb lo ignora per *k*-1 volte e ferma il programma subito prima della *k*-esima esecuzione. Se il programma termina prima, l'esperimento è "non iniettato" e viene scartato. Succede di rado, per un dettaglio di valgrind: le poche istruzioni con cui il programma salta a una funzione della libreria C (la *PLT*) vengono contate insieme all'istruzione `call` che la chiama, quindi per quelle `call` il profilo indica qualche esecuzione in più di quelle reali. Gli esperimenti rimasti sono comunque distribuiti in modo uniforme su tutte le esecuzioni reali;
3. **guasto**:
   - bit-flip: legge il registro, inverte il bit con uno XOR e lo riscrive (`set $rax = ...`);
   - salto invertito: esegue il salto con `stepi`, guarda dove è andato e sposta il *program counter* (`$pc`) sull'altro successore: la destinazione del salto se il salto non è stato preso, l'istruzione successiva se è stato preso;

4. **prosecuzione**: toglie il breakpoint e lascia correre il programma. Un piccolo processo di guardia (un `sleep` seguito da `kill -KILL`) uccide il programma se non termina entro il tempo limite: è un *hang*;
5. **risultato**: stampa una riga `FI-RESULT` con tre informazioni: se il guasto è stato iniettato, il codice d'uscita e l'eventuale segnale che ha fermato il programma (per esempio `SIGSEGV`).

9.4 Classificare l'esito

`campaign.py` legge la riga `FI-RESULT` e i due file di output e classifica l'esperimento, in quest'ordine:

- `hang` se il programma è stato ucciso dal processo di guardia: dopo il guasto non è terminato entro 20 volte il suo tempo normale, e comunque almeno 1 secondo;
- `crash` se è stato fermato da un altro segnale;
- `detected_data` se su stderr c'è `FAULT_DETECTED: DataCorruption`;
- `detected_cfc` se su stderr c'è `FAULT_DETECTED: SigMismatch`;
- `masked` se l'output è identico a quello di riferimento e il codice d'uscita è 0;
- `sdc` altrimenti (output diverso, oppure il programma stesso si è accorto dell'errore e ha stampato `FAIL`).

Ogni esperimento diventa una riga di `raw.csv.gz`. Per esempio, dai risultati di `aes128` con EDDI:

```
application,config,model,id,address,occurrence,target,bit,outcome,exit,signal,time_ms
aes128,EDDI,reg,0,881f,3096,rax,17,masked,0,-,505
aes128,EDDI,reg,7,8974,3237,rsp,3,crash,-1,SIGSEGV,501
aes128,EDDI,reg,47,8939,7700,rax,4,detected_data,2,-,1239
```

Il primo esperimento ha invertito il bit 17 di `rax` prima della 3096ª esecuzione dell'istruzione `0x881f`, senza effetti visibili (*masked*). Il secondo ha invertito il bit 3 di `rsp`, il puntatore allo stack: il programma è stato fermato dal segnale `SIGSEGV` (*crash*). Nel terzo il valore sbagliato di `rax` è stato fermato da un controllo di EDDI: il programma ha stampato `FAULT_DETECTED: DataCorruption` ed è uscito con codice 2 (*detected_data*). L'ultima colonna è la durata dell'esperimento, gdb compreso.

9.5 Quanto costa un esperimento

Un esperimento dura in media circa 0,4 secondi (376 ms; la mediana è 283 ms), con quattro esperimenti in parallelo. La maggior parte del tempo è di gdb: l'avvio (circa un decimo di secondo) e le fermate sul breakpoint, perché per ignorare i primi k-1 passaggi gdb ferma comunque il programma ogni volta e lo fa ripartire (circa un decimo di millisecondo per passaggio, che nei cicli sono migliaia). Gli esperimenti sono indipendenti, quindi `campaign.py` ne esegue quattro alla volta, uno per core.

9.6 Cosa non facciamo

- iniettiamo solo mentre il programma esegue **il proprio codice**, mai nella libreria C (che ASPIS non protegge);
- non tocchiamo i registri **SSE** (`xmm0` …), dove stanno i valori in virgola mobile: per `fft` e `pid_control` i guasti nei calcoli in virgola mobile sono rappresentati solo quando colpiscono indirizzi, indici o interi;
- un solo guasto per esecuzione, niente guasti in memoria.

10. La campagna e la statistica

10.1 Il file di configurazione

`fault_injection/campaign.toml` descrive la campagna:

```toml
[campaign]
root = ".."                                # i sorgenti sono relativi a ASPIS-Project/
work = "~/aspis-project/campaign"          # eseguibili, profili, risultati di ogni gruppo
results = "../results/fault_injection"     # metrics.csv, outcomes.csv, raw.csv.gz
aspis = "~/aspis-project/ASPIS"            # ASPIS con le nostre patch
llvm_bin = "~/aspis-project/toolchain/LLVM-21.1.8-Linux-X64/bin"
tmp = "/dev/shm/aspis-fi"                  # file di output degli esperimenti
models = ["reg", "branch"]
injections = 500                           # per applicazione x configurazione x modello
seed = 2026
timeout_factor = 20                        # dopo il guasto: hang = 20 volte il tempo normale...
timeout_min_ms = 1000                      # ...e almeno 1 secondo

[[config]]                                 # la prima è il riferimento
name = "baseline"
flags = "--no-dup --no-cfc"
...                                        # EDDI, CFCSS, RASM, EDDI+CFCSS, EDDI+RASM

[[application]]
name = "fft"
source = "benchmarks/fft.c"
flags = "-lm"
...
```

I percorsi in `[campaign]` vanno adattati alla propria macchina.

10.2 Cosa fa `campaign.py`

```
python3 fault_injection/campaign.py fault_injection/campaign.toml --jobs 4
```

1. **Compila** ogni applicazione con ogni configurazione (`aspis.sh` con le opzioni della configurazione), ognuna nella sua cartella `build/<applicazione>/<configurazione>/`. `aspis.sh` viene eseguito *dentro* quella cartella perché EDDI scrive un file di lavoro (`compiled_eddi_functions.csv`) nella cartella corrente: due compilazioni in parallelo nella stessa cartella si sovrascriverebbero il file a vicenda.
2. **Misura** la dimensione del codice (sezione `.text`, con `llvm-size`) e il numero di controlli inseriti da EDDI (la riga `Comparison Counter` stampata da `aspis.sh`).
3. **Fa le golden run** (capitolo 9.1) e controlla che tutte le configurazioni stampino esattamente lo stesso output della `baseline`: una protezione non deve cambiare il risultato.
4. Scrive `metrics.csv`: per ogni applicazione e configurazione, dimensione del codice, controlli, istruzioni eseguite (contate da valgrind).
5. **Prepara gli esperimenti** di ogni gruppo (applicazione, configurazione, modello): 500 istanti scelti come nel capitolo 9.2, con un generatore di numeri casuali il cui seme è calcolato con SHA-256 dai nomi (`"2026/aes128/EDDI/reg"` → i primi 15 caratteri esadecimali dell'hash). Il seme è sempre lo stesso, quindi la campagna è **riproducibile**.

6. **Lancia gli esperimenti** con gdb (capitolo 9.3), quattro alla volta, e li classifica (9.4). Il tempo limite per gli *hang* è 20 volte il tempo normale del programma, e almeno 1 secondo: per i nostri programmi, che durano pochi millisecondi, vale sempre 1 secondo.
7. **Aggrega**: scrive `raw.csv.gz` (una riga per esperimento) e `outcomes.csv` (gli esiti per applicazione, configurazione e modello, con la percentuale di SDC e il suo intervallo di confidenza).

Ogni gruppo salva i suoi risultati su file appena finito (scrivendo prima un file `.part` e rinominandolo alla fine): se la campagna si interrompe, rilanciandola rifà solo quello che manca.

I numeri della nostra campagna: 4 applicazioni × 6 configurazioni × 2 modelli × 500 = **24.000 esperimenti**, più **4000** di EDDI+RASM con ASPIS senza la correzione di RASM (4 applicazioni × 2 modelli × 500). Su una macchina virtuale con 4 core la campagna principale ha richiesto circa 38 minuti (compilazioni e golden run pochi secondi, il resto esperimenti), quella senza correzione circa 6. Undici bit-flip su 12.000 sono risultati "non iniettati" (capitolo 9.3) e non vengono contati: per questo alcune configurazioni hanno 1995-1999 esperimenti invece di 2000.

10.3 Percentuali e intervalli di confidenza

Ogni percentuale della campagna è una **stima**: abbiamo scelto 2000 guasti a caso (500 per applicazione) tra tutti quelli possibili, e la percentuale osservata `p = k / n` (`k` SDC su `n` esperimenti) è vicina, ma non uguale, alla vera probabilità. L'**intervallo di confidenza al 95%** è un intervallo costruito in modo che, se si ripetesse la campagna molte volte, conterrebbe la vera probabilità nel 95% dei casi.

La formula più nota (intervallo "di Wald") è `p ± 1,96 · sqrt(p(1-p)/n)`, ma funziona male quando `p` è vicina a 0, proprio il nostro caso: con `k = 0` dà l'intervallo `[0, 0]`, come se fossimo sicuri che la probabilità sia esattamente zero. Usiamo quindi l'**intervallo di Wilson**, che si comporta bene anche agli estremi:

$$\text{intervallo} = \frac{p + z^2/(2n) \;\pm\; z \cdot \sqrt{\,p(1-p)/n + z^2/(4n^2)\,}}{1 + z^2/n} \qquad \text{con } z = 1{,}96$$

Esempi dalla nostra campagna:

	k / n	p	intervallo di Wilson al 95%
EDDI, bit-flip nei registri	0 / 2000	0%	0,00% – 0,19%
EDDI+RASM, salti invertiti	0 / 2000	0%	0,00% – 0,19%
baseline, bit-flip nei registri	42 / 1995	2,11%	1,56% – 2,83%

Il primo esempio si legge: "non abbiamo visto nessuna SDC su 2000 bit-flip; la vera probabilità è, con il 95% di confidenza, sotto lo 0,19%". Quando due intervalli non si sovrappongono (1,56-2,83% contro 0,00-0,19%), la differenza non è dovuta al caso.

10.4 La media geometrica

Per riassumere il costo su quattro applicazioni usiamo la **media geometrica** dei rapporti, `(r1 · r2 · r3 · r4)^(1/4)`, non quella aritmetica. Il motivo: i rapporti si moltiplicano, non si sommano. Se un programma raddoppia (2x) e un altro dimezza (0,5x), la media aritmetica dice 1,25x, quella geometrica 1x, che è la risposta giusta ("in media non cambia nulla"). Per EDDI i rapporti delle istruzioni eseguite sono 3,68 (`aes128`), 3,63 (`fft`), 3,56 (`huffman`) e 4,12 (`pid_control`): la media geometrica è 3,74.

10.5 Gli script di analisi

- `analyze.py` legge i risultati della campagna e i log del sistema di test e scrive le tabelle (`report/tables/*.md`, in formato Markdown) e le figure (`report/figures/*.svg` e `.png`, con matplotlib) che `report/build.py` inserisce nel report;
- `sdc_sites.py` (capitolo 8.3) mostra dove sono avvenute le SDC di una configurazione: le istruzioni colpite, quante volte, e i successori dei salti;
- `check_jumps.py` (capitolo 11.3) conta, dai profili delle golden run, quanti dei salti condizionali eseguiti appartengono ai controlli di ASPIS.

11. I risultati, spiegati

11.1 Il costo delle protezioni

Configurazione	Dimensione del codice	Istruzioni eseguite	Controlli (EDDI)
baseline	1,00x	1,00x	0
EDDI	9,27x	3,74x	1283
CFCSS	1,58x	1,65x	0
RASM	1,99x	2,33x	0
EDDI+CFCSS	13,90x	5,46x	1283
EDDI+RASM	14,80x	5,95x	1283

(medie geometriche sulle quattro applicazioni, rispetto alla `baseline`; 1283 è il totale dei controlli inseriti nelle quattro applicazioni.)

Cosa dire:

- **EDDI costa circa 3,7 volte** le istruzioni (da 3,56x per `huffman` a 4,12x per `pid_control`): ogni istruzione è eseguita due volte, e ogni controllo aggiunge confronti e un salto;
- **il codice cresce molto più dell'esecuzione** (9,3x): ogni funzione esiste fino in quattro versioni (`f`, `f_ret`, `f_dup`, `f_ret_dup`), anche quelle che alla fine non vengono più chiamate restano nell'eseguibile, ogni controllo spezza un blocco in più blocchi, e senza ottimizzazioni ogni valore passa dallo stack;
- **le tecniche del flusso costano meno** (1,65x CFCSS, 2,33x RASM): aggiungono poche istruzioni per blocco. RASM costa più di CFCSS perché aggiorna la firma due volte per blocco (all'ingresso e prima dell'uscita, con la `select`);
- **combinate, i costi si sommano**: EDDI+RASM esegue circa 6 volte (5,95x) le istruzioni dell'originale.

Il costo è misurato in **istruzioni eseguite** (contate dal profilo), non in tempo: i programmi durano pochi millisecondi, troppo poco per misure di tempo affidabili, mentre il numero di istruzioni è esatto e ripetibile.

11.2 Bit-flip nei registri

Configurazione	n	masked	rilevati (dati)	rilevati (CFC)	crash	hang	**SDC**	intervallo 95%
baseline	1995	83,6	0,0	0,0	14,3	0,0	**2,11**	1,56–2,83
EDDI	2000	83,3	2,9	0,0	13,9	0,0	**0,00**	0,00–0,19
CFCSS	1997	85,1	0,0	3,0	11,1	0,1	**0,75**	0,46–1,24
RASM	1998	85,4	0,0	2,8	11,0	0,0	**0,80**	0,49–1,30
EDDI+CFCSS	2000	83,7	1,3	2,3	12,7	0,0	**0,05**	0,01–0,28
EDDI+RASM	1999	83,8	1,9	1,6	12,6	0,0	**0,20**	0,08–0,51

(percentuali sugli n guasti di ogni riga: 2000 meno i pochi "non iniettati" del capitolo 9.3.)

Cosa dire:

- **la maggior parte dei bit-flip non fa nulla** (83,6% *masked*): in un dato istante molti registri non contengono valori che verranno usati, o vengono sovrascritti prima di essere letti;
- **il 14% fa andare in crash il programma**: un bit cambiato in un indirizzo porta a leggere o scrivere fuori dalla memoria valida. Nella `baseline` l'89% dei crash viene da un bit-flip in `rsp` (47%) o in `rbp` (42%), i due pun-

tatori allo stack: con `-O0` ogni variabile si legge e si scrive attraverso di loro;

- **il 2,11% produce un risultato sbagliato in silenzio** (42 casi su 1995): sono questi i guasti che contano;
- **con EDDI non ne resta nessuno** su 2000: con il 95% di confidenza la sua percentuale di SDC è sotto lo 0,19%, più di dieci volte meno. I suoi controlli scattano solo nel 2,9% dei casi, ma sono proprio i casi pericolosi: un valore sbagliato viene fermato prima di essere scritto in memoria, passato a una funzione o usato in un salto;
- **CFCSS e RASM da soli aiutano poco** (0,75% e 0,80%): una firma vede solo i bit-flip che cambiano il percorso del programma, non quelli che cambiano un valore;
- **combinando EDDI con una tecnica del flusso restano 1 e 4 SDC su 2000** (0,05% con CFCSS, 0,20% con RASM): con così pochi casi queste percentuali non si distinguono da quella di EDDI da solo, perché gli intervalli si sovrappongono. Non vuol dire che aggiungere RASM peggiori EDDI: è il caso.

11.3 Salti condizionali invertiti

Configurazione	masked	rilevati (dati)	rilevati (CFC)	crash	**SDC**	intervallo 95%
baseline	20,7	0,0	0,0	0,0	**79,30**	77,47–81,02
EDDI	4,1	77,2	0,0	0,0	**18,75**	17,10–20,52
CFCSS	8,5	0,0	62,5	0,1	**29,05**	27,10–31,08
RASM	0,0	0,0	99,3	0,0	**0,70**	0,42–1,17
EDDI+CFCSS	2,5	36,3	53,5	0,0	**7,70**	6,61–8,95
EDDI+RASM	0,0	32,9	67,1	0,0	**0,00**	0,00–0,19

(2000 guasti per riga; nessun salto invertito ha prodotto un *hang*, per questo la colonna manca.)

Cosa dire:

- **i salti invertiti sono i guasti peggiori**: nei programmi senza protezione il 79,3% produce un risultato sbagliato, perché il risultato dipende da quante volte si ripetono i cicli e da quale ramo si prende;
- **EDDI e CFCSS rilevano quasi solo i salti dei loro stessi controlli.** Abbiamo misurato dai profili (con `fault_injection/check_jumps.py`) quanti dei salti condizionali eseguiti appartengono ai controlli, cioè hanno come destinazione il blocco che chiama il gestore d'errore: il **76,3%** con EDDI e il **62,4%** con CFCSS. Se si inverte uno di questi salti, il programma va nel blocco d'errore e il guasto risulta "rilevato". Infatti le percentuali di rilevamento (77,2% e 62,5%) sono molto vicine. Durante la verifica finale abbiamo anche classificato uno per uno i salti colpiti: **tutti** i guasti rilevati da CFCSS e il 98% di quelli rilevati da EDDI (1517 su 1543) erano inversioni di salti dei controlli. Quando invece si inverte un salto **del programma**:
  - EDDI non se ne accorge, perché confronta le due copie della condizione *prima* del salto, ma poi esegue un solo salto: entrambe le copie dei dati vanno dalla parte sbagliata e restano coerenti tra loro;
  - CFCSS non se ne accorge, perché il successore sbagliato è comunque un successore lecito (capitolo 4.3);

  e restano il 18,75% e il 29,05% di SDC;
- **RASM rileva il 99,3%**: anche con RASM i salti dei controlli sono la maggioranza (67,4%), ma RASM rileva anche le inversioni dei salti del programma, perché la firma viene aggiornata con la stessa condizione del salto (capitolo 4.4);
- **le 14 SDC rimaste a RASM (0,70%) vengono tutte da un solo salto** di `pid_control`, nella funzione `load_at`:

```
static float load_at(int k) {
    return (k >= 1200 && k < 1300) ? 1.5f : 0.0f;
}
```

Nell'IR questa funzione è un solo blocco: la condizione è calcolata senza salti e il `? :` è una `select` tra due `float`. RASM non ha nessun salto da proteggere. Ma il back-end x86, senza ottimizzazioni, traduce la `select` con un `jne`. Lo si vede con `sdc_sites.py`: i due successori del salto caricano due valori diversi in `xmm0` (nel disassemblato la funzione si chiama `__unnamed_8`, perché il passo `strip` della pipeline toglie i nomi delle funzioni `static`; la si riconosce dai confronti con `0x4b0` = 1200 e `0x514` = 1300). Nessuna tecnica che lavora sull'IR può vedere un salto creato dopo, dal back-end. È un limite interessante da citare all'orale;

- **EDDI+RASM non lascia nessuna SDC**: le due tecniche si coprono a vicenda. RASM vede i salti del programma andati dalla parte sbagliata; EDDI vede il caso di `load_at`, perché duplica anche la `select`: il back-end crea due salti, uno per copia, e se uno dei due sbaglia le due copie del valore differiscono e il controllo successivo se ne accorge (sono 11 dei 26 salti del programma rilevati da EDDI da solo).

11.4 Il bug di RASM nei numeri

Modello di guasto	SDC prima	SDC dopo	rilevati (CFC) prima	rilevati (CFC) dopo
bit-flip nei registri	0,15%	0,20%	0,4%	1,6%
salti invertiti	**9,70%**	**0,00%**	36,4%	67,1%

Prima della correzione, EDDI+RASM lasciava il 9,7% di SDC sui salti invertiti: peggio di RASM da solo (0,70%), anche se aveva in più tutti i controlli di EDDI. È stato questo controsenso a far partire l'indagine (capitolo 8.3): tutte le 194 SDC erano su salti verso blocchi `VerificationBB` di EDDI dove RASM reimpostava la firma senza controllarla. Dopo la correzione i controlli di RASM rilevano il 67,1% dei salti invertiti invece del 36,4%, e non resta nessuna SDC. Sui bit-flip nei registri la correzione non cambia le SDC (3 e 4 casi su 2000, una differenza dovuta al caso).

11.5 Per applicazione

SDC dei bit-flip nei registri (%, 500 guasti per cella):

Applicazione	baseline	EDDI	CFCSS	RASM	EDDI+CFCSS	EDDI+RASM
aes128	2,0	0,0	0,8	0,8	0,2	0,4
fft	2,2	0,0	0,8	0,6	0,0	0,2
huffman	2,0	0,0	0,6	1,0	0,0	0,0
pid_control	2,2	0,0	0,8	0,8	0,0	0,2

SDC dei salti invertiti (%):

Applicazione	baseline	EDDI	CFCSS	RASM	EDDI+CFCSS	EDDI+RASM
aes128	88,8	21,8	31,0	0,0	8,6	0,0
fft	68,4	13,6	26,2	0,0	4,6	0,0
huffman	90,0	24,6	30,0	0,0	9,4	0,0
pid_control	70,0	15,0	29,0	2,8	8,2	0,0

Con 500 guasti per cella le percentuali delle singole applicazioni sono poco precise: per i bit-flip nei registri il 2,0% di `aes128` sono 10 casi, con un intervallo al 95% che va dall'1,1% al 3,6%. Le quattro applicazioni senza protezione hanno percentuali simili (dal 2,0% al 2,2%) e EDDI le porta tutte a zero. Per i salti invertiti, invece, le differenze

nella `baseline` sono reali (dal 68,4% di `fft` al 90,0% di `huffman`): dipendono da quanti salti del programma, se invertiti, cambiano davvero il risultato. Le protezioni però si comportano allo stesso modo in tutte e quattro le applicazioni; l'unica eccezione è quella già spiegata, le SDC di RASM in `pid_control`.

11.6 Il messaggio in tre frasi

1. La duplicazione dei dati (EDDI) protegge bene dai bit-flip nei registri (nessuna SDC su 2000) ma costa molto (3,7 volte le istruzioni) e non vede i salti andati dalla parte sbagliata.
2. Il controllo del flusso a firme vede quei salti solo se la firma dipende dalla direzione del salto (RASM sì, CFCSS no); nessuna tecnica sull'IR vede i salti creati dal back-end.
3. Combinare le due famiglie dà la protezione migliore, a patto che le due trasformazioni non si ostacolino: la fault injection ha trovato proprio un caso in cui lo facevano (il bug di RASM).

Parte III. Pratica ed esame

12. L'integrazione in ASPIS

Il terzo obiettivo è portare le applicazioni nel branch principale di ASPIS come test di regressione. Il lavoro è pronto in `aspis-integration/`:

File	Contenuto
`patches/0001-fix-RASM-RACFED-check-...`	la correzione del bug di RASM (`RASM.cpp`, `RACFED.cpp`, una riga ciascuno più un commento)
`patches/0002-test-add-the-applications-...`	le quattro applicazioni in `testing/tests/c/applications/` e le loro voci in `testing/config/tests.toml`
`bug_report.md`	il bug di EDDI, con riproduttore, causa e possibile correzione, pronto da aprire come *issue*
`reproducers/string_literal.c`	il riproduttore di cinque righe

Cos'è una patch di git. È un file di testo che contiene un commit: autore, data, messaggio e le differenze (*diff*) dei file. Si crea con `git format-patch` e si applica, ricreando lo stesso commit, con `git am`:

```
# come le abbiamo create, dal nostro clone di ASPIS con i 2 commit
git format-patch b66a090..HEAD -o patches/
# come si applicano su un clone pulito di ASPIS
git checkout b66a090
git am patches/*.patch
```

Ogni messaggio di commit spiega il problema e la correzione, così chi deve accettare le modifiche può valutarle una per una. Abbiamo verificato che le patch si applicano senza conflitti su `b66a090` e che il risultato è identico alla versione di ASPIS usata per tutti gli esperimenti.

Come proporle (quando deciderete di farlo). Il modo normale è una *pull request* su GitHub: si fa un *fork* di `HEAPLab/ASPIS`, si applicano le patch su un branch del fork, lo si pubblica e si apre la pull request verso il repository originale, descrivendo il lavoro (si può rimandare al report). Il bug di EDDI si segnala aprendo una *issue* con il testo di `bug_report.md`. Conviene parlarne prima con i supervisori, che sono anche gli sviluppatori di ASPIS: potrebbero preferire ricevere la correzione separata dai test, o discuterne durante la chiamata. Per ora **non è stato inviato nulla**.

13. Rifare tutto da zero

Requisiti: Linux x86-64 (anche WSL2 o una macchina virtuale), Python 3.11 o successivo con `pytest`, `pytest-timeout` e `matplotlib`, CMake, Ninja, gcc, gdb e valgrind; per rigenerare il PDF del report anche `pandoc` e Node.js con Playwright.

```
# 1. LLVM 21.1.8 già compilato (circa 2 GB da scaricare, 11 GB estratto)
curl -LO https://github.com/llvm/llvm-project/releases/download/llvmorg-21.1.8/LLVM-21.1.8-Linux-
X64.tar.xz
tar xf LLVM-21.1.8-Linux-X64.tar.xz            # <llvm> = la cartella estratta

# 2. ASPIS con le nostre patch (circa 1 minuto)
git clone --recursive https://github.com/HEAPLab/ASPIS && cd ASPIS
git checkout b66a090
git am <repo>/ASPIS-Project/aspis-integration/patches/*.patch
cmake -B build -G Ninja -DLLVM_DIR=<llvm>/lib/cmake/llvm && cmake --build build

# 3. i test di ASPIS (circa 9 minuti)
#    prima scrivere llvm_bin = "<llvm>/bin" in testing/config/llvm.toml
cd testing && pytest test.py -v > harness_fixed.log && cd ..

# 4. un programma a mano, per vedere cosa succede
./aspis.sh --llvm-bin <llvm>/bin --eddi --rasm <repo>/ASPIS-Project/benchmarks/aes128.c \
    -o aes128 --build-dir /tmp/aes --no-cleanup && ./aes128
less /tmp/aes/out.ll                           # l'IR finale

# 5. la campagna (circa 40 minuti con 4 core); scrive i risultati in results/
#    adattare i percorsi in fault_injection/campaign.toml, poi:
python3 <repo>/ASPIS-Project/fault_injection/campaign.py <repo>/ASPIS-
Project/fault_injection/campaign.toml

# 6. tabelle, figure e report
cd <repo>/ASPIS-Project
python3 fault_injection/analyze.py --campaign results/fault_injection \
    --before results/fault_injection/before_fix --out report \
    --harness "first version, master=results/compatibility/harness_master_first_version.log.gz" \
    --harness "final, master=results/compatibility/harness_master.log.gz" \
    --harness "final, with patches=results/compatibility/harness_fixed.log.gz"
python3 report/build.py
```

Per la parte “prima della correzione di RASM” serve un secondo ASPIS, senza patch (`b66a090` così com’è), con cui lanciare la campagna con la sola configurazione EDDI+RASM (con `work` e `results` diversi: `results/fault_injection/before_fix`). Per il log dei test su `master` si lancia `pytest` sull’ASPIS senza patch, aggiungendo le nostre voci con `--tests-file config/tests.toml <repo>/ASPIS-Project/aspis-integration/flc_tests.toml`; per quello della prima versione delle applicazioni serve il codice di quella versione (commit `7074cd6` del nostro repository).

Le golden run e gli esperimenti sono deterministici (stessi semi, indirizzi fissi): rifacendo la campagna con gli stessi eseguibili si ottengono gli stessi esperimenti e gli stessi esiti. Ricompilando le applicazioni, però, l’eseguibile prodotto da EDDI può cambiare un poco, perché EDDI visita le istruzioni in un ordine che dipende dagli indirizzi di memoria del compilatore (usa un `std::set` di puntatori); quindi anche i numeri possono variare di poco.

14. Mappa dei file

Tutti i percorsi sono relativi a `ASPIS-Project/`.

File	Cosa contiene
`README.md`	presentazione del progetto e indice
`benchmarks/aes128.c`, `fft.c`, `huffman.c`, `pid_control.c`	le quattro applicazioni (capitolo 6)
`aspis-integration/README.md`	come applicare le patch
`aspis-integration/patches/0001-*.patch`, `0002-*.patch`	le patch per ASPIS (capitolo 12)
`aspis-integration/flc_tests.toml`	le voci dei test aggiunte a `tests.toml`
`aspis-integration/bug_report.md`	il bug report di EDDI (capitolo 8.2)
`aspis-integration/reproducers/string_literal.c`	il riproduttore del bug di EDDI; `handlers.h` contiene i due gestori d'errore
`fault_injection/gdb_inject.py`	un esperimento, eseguito dentro gdb (capitolo 9)
`fault_injection/campaign.py`, `campaign.toml`	la campagna: compilazioni, golden run con valgrind, esperimenti, risultati (capitolo 10)
`fault_injection/analyze.py`	tabelle e figure del report
`fault_injection/sdc_sites.py`	dove sono avvenute le SDC (capitolo 8.3)
`fault_injection/check_jumps.py`	quanti dei salti eseguiti appartengono ai controlli (capitolo 11.3)
`results/README.md`	descrizione dei risultati
`results/compatibility/harness_master_first_version.log.gz`	il log dei test con la prima versione delle applicazioni
`results/compatibility/harness_master.log.gz`, `harness_fixed.log.gz`	i log dei test con la versione finale, senza e con le patch
`results/fault_injection/metrics.csv`	costo: dimensione, controlli, istruzioni eseguite
`results/fault_injection/outcomes.csv`	esiti per applicazione, configurazione, modello
`results/fault_injection/raw.csv.gz`	i 24.000 esperimenti, uno per riga
`results/fault_injection/before_fix/`	gli stessi file per EDDI+RASM senza la correzione di RASM
`report/report.md`	il testo del report (Markdown, con i segnaposto `{{table:...}}` per le tabelle)
`report/build.py`	genera `report.html` e `report.pdf` (pandoc + Chromium) e la versione web
`report/style.css`	lo stile del report
`report/tables/`, `report/figures/`	tabelle e figure generate da `analyze.py`
`report/report.pdf`	**il report da consegnare**

Nella radice del repository: `README.md`, la proposta del progetto (`FLC Project Proposals AY 202627.txt`), le indicazioni dei supervisori (`Buongiorno,.txt`) e, fuori dalla cartella del progetto, questa guida (`guida/`).

15. Cosa è stato tolto (le versioni precedenti)

Il progetto è passato per quattro versioni, tutte nella storia del repository:

Versione	Commit	Cosa conteneva
prima (completa)	`e1d7e8b`	22 programmi, 23 bug corretti, campagna di 240.900 esperimenti, report di 19 pagine
seconda	`7074cd6`	le quattro applicazioni attuali (nella prima versione delle stampe), tre bug corretti: due in EDDI e quello di RASM, un iniettore scritto da noi
terza	`0609127`	le applicazioni che evitano il bug di EDDI, la sola correzione di RASM, ancora l'iniettore scritto da noi
quarta (attuale)	`main`	come la terza, ma i guasti si iniettano con gdb e valgrind

Per vedere una versione precedente:

```
git show e1d7e8b:ASPIS-Project/report/report.pdf > report-completo.pdf
git checkout 7074cd6      # tutto il repository com'era (poi: git checkout main)
```

o su GitHub, aprendo il commit e scegliendo "Browse files".

La prima versione conteneva, in più rispetto a quella attuale:

- **22 programmi** invece di 4: altre 4 applicazioni (un dizionario su albero binario di ricerca, un piccolo compilatore di espressioni con macchina virtuale, un risolutore LU, un filtro di Sobel) e 14 *microbenchmark*, ognuno dedicato a una caratteristica del C (puntatori a funzione richiamati dalla libreria, funzioni variadiche, struct passate per valore, array a lunghezza variabile, `setjmp` / `longjmp`, campi di bit, …);
- **23 bug corretti**, in una serie di 19 commit: tra gli altri, la deduzione dei tipi che non terminava con le struct ricorsive (il compilatore si bloccava), i puntatori a funzione passati a `qsort`, le inizializzazioni delle globali duplicate, le espressioni costanti annidate, `memmove`, le funzioni variadiche, gli array a lunghezza variabile, REDDI, i campi di bit, `setjmp` / `longjmp` nelle tecniche a firme, inter-RASM con le funzioni chiamate dalla libreria. Con quelle correzioni passavano tutte le 1210 combinazioni dei 22 programmi (contro 435 sul branch principale) e 23 combinazioni escluse dalle `black_list` di ASPIS;
- due **strumenti di test** propri, più robusti del sistema di test ufficiale;
- un **iniettore** più veloce, che usava i *breakpoint hardware* del processore (con la chiamata di sistema `perf_event_open`) invece di `int3`, e un terzo modello di guasto (salto a un indirizzo casuale);
- una **campagna di 240.900 esperimenti**: 8 applicazioni, 11 configurazioni (anche sEDDI, FDSC, inter-RASM, RACFED, EDDI+RACFED), 3 modelli di guasto, e un'analisi del rapporto costo/beneficio (*mean work to failure*).

La seconda e la terza versione avevano un iniettore scritto da noi, `fault_injection/injector/fi.c`: circa 560 righe di C che usavano direttamente `ptrace`, la chiamata di sistema dei debugger, per contare le istruzioni eseguendo il programma un passo alla volta, fermarlo con un breakpoint scritto a mano nel codice (l'istruzione `int3`) e decodificare i salti condizionali dai loro byte. Era più veloce (circa 60 ms per esperimento, 48.000 esperimenti in circa 20 minuti), ma era la parte più tecnica del progetto. La versione attuale fa le stesse cose con gdb e valgrind, che lo fanno per noi.

La seconda versione correggeva, oltre a RASM, due bug di EDDI: quello della copia dopo le chiamate di libreria (che ora evitiamo, capitolo 8.2) e un bug dei controlli che leggevano attraverso puntatori nulli (per esempio con `realloc(NULL, n)`), che le nostre applicazioni non attivano. Erano circa 160 righe di C++ dentro `EDDI.cpp`.

Le parti tolte richiederebbero a due studenti, a mano, molte più ore di quante ne valga un progetto da 30% di un esame da 5 CFU, e sono le più difficili da studiare e da difendere all'orale. Le correzioni restano comunque utili: se un giorno vorrete proporle ad ASPIS, sono nei commit indicati (`ASPIS-Project/aspis-integration/patches/`).

16. La chiamata con i supervisori

I supervisori hanno scritto: *"Quando siete pronti facciamo una call dove ci spiegate quel che avete fatto. In più, dovete redarre un report che ci serve come reference per la valutazione"*. Per la chiamata c'è una presentazione di 17 slide, in inglese come il report: https://claude.ai/artifact/R4PFNt1VCATSuovbDMCFzV. Si può proiettare dal browser o scaricare in PDF o PowerPoint.

Ogni slide ha delle **note per chi parla**, in italiano, con quello che c'è da dire: si vedono nella modalità presentatore. Attenzione: le note sono visibili a chiunque apra la presentazione, quindi se la condividete con i supervisori le leggeranno anche loro.

Durata e divisione. Circa 15 minuti, poi le domande. Una divisione naturale: uno presenta la prima metà (slide 1-6: progetto, ASPIS, applicazioni, test di compatibilità, bug di EDDI), l'altro la seconda (slide 7-13: iniettore, campagna, risultati, bug di RASM), e le ultime quattro (integrazione, limiti, intelligenza artificiale, domande) le fate insieme.

Il consiglio più importante. I supervisori sono gli sviluppatori di ASPIS: conoscono le tecniche molto meglio di noi. Non serve spiegare loro EDDI o RASM; serve mostrare che sapete cosa avete fatto, perché, e cosa avete trovato. Le parti su cui saranno più curiosi sono i due bug (slide 6, 12 e 13) e la domanda finale su come ricevere le patch.

Di seguito, slide per slide, il messaggio da far passare e i numeri da ricordare. Tutti i numeri sono gli stessi del report e dei capitoli 7-11.

16.1 Parte 1: il progetto (slide 1-2)

1. Copertina. Presentarsi e dare la scaletta: le applicazioni, i due bug, la fault injection, cosa portiamo in ASPIS, i limiti, l'uso dell'IA, tre domande per loro.

2. Tre obiettivi. La proposta chiedeva di proteggere applicazioni C con ASPIS, di verificare la protezione con la fault injection e di integrare le applicazioni in ASPIS come test di regressione. Per ognuno, una riga di risultato: 55 combinazioni su 55 per ogni applicazione; 24.000 esperimenti più 4000, con i guasti iniettati da gdb; due patch e un bug report, non ancora inviati.

16.2 Parte 2: le applicazioni e i test (slide 3-6)

3. ASPIS. Un richiamo veloce, solo per mostrare che sapete come funziona: pass sull'IR, duplicazione dei dati (EDDI e varianti), firme sul flusso (CFCSS, RASM, RACFED), compilazione a `-O0` senza ottimizzazioni dopo i pass (capitoli 4-5).

4. Le applicazioni. Avevano lasciato a noi la scelta e chiesto di poterne discutere: quattro applicazioni scritte da noi, che si controllano da sole (stampano `SUCCESS` o `FAIL`), deterministiche, brevi (meno di un milione di istruzioni), valide in C e in C++, con licenza MIT perché possano entrare nel repository di ASPIS (capitolo 6). Sono anche la base della domanda finale "ne bastano quattro?".

5. Compatibilità. Il sistema di test ufficiale, 55 combinazioni per programma. Con la prima versione delle applicazioni passavano 100 combinazioni su 220: EDDI, sEDDI e FDSC andavano in crash su tutte e quattro. Con la versione finale passano tutte e 220, sia su `master` sia con le patch. I 48 test esistenti danno gli stessi risultati con e senza patch (2453 superati, 33 falliti, 154 saltati). Se chiedono dei 33 falliti: sono tutti di `cpp_complex`, che non compila nemmeno con `clang++` sulla nostra macchina (capitolo 7).

6. Bug di EDDI. Il riproduttore di cinque righe, la causa (dopo una chiamata di libreria EDDI copia sempre 8 byte dall'originale alla copia di ogni argomento puntatore, e con una stringa letterale scrive in memoria di sola lettura), cosa abbiamo fatto (aggirato nelle applicazioni, documentato nel bug report, non corretto dentro ASPIS). Se chie-

dono perché non l'abbiamo corretto: la correzione vera richiede di riscrivere `fixFuncValsPassedByReference`, e preferiamo proporla a loro con il bug report piuttosto che toccare EDDI da soli (capitolo 8.2).

16.3 Parte 3: la fault injection (slide 7-13)

7. L'iniezione con gdb. Non abbiamo scritto un iniettore: valgrind (`callgrind`) conta le esecuzioni di ogni istruzione nella golden run, poi in ogni esperimento gdb ferma il programma in un istante a caso (un breakpoint con *ignore count*), lo script inietta il guasto (un bit di un registro con `set`, o il salto mandato dall'altra parte cambiando `$pc`) e la campagna classifica l'esito confrontando l'output con quello di riferimento. Ricordare la definizione di SDC: un risultato sbagliato senza nessun segnale d'errore (capitolo 9). Se chiedono perché non un iniettore nostro: gdb fa già tutto, ed è più facile da verificare; il prezzo è il tempo.

8. La campagna. 4 applicazioni × 6 configurazioni × 2 modelli × 500 = 24.000 esperimenti, più 4000 di EDDI+RASM senza la correzione. Semi fissi, ASLR disattivato, intervalli di Wilson al 95%, circa 45 minuti in tutto su 4 core (capitolo 10).

9. Il costo. EDDI esegue 3,74 volte le istruzioni e ha il codice 9,27 volte più grande; CFCSS 1,65x e RASM 2,33x; EDDI+RASM 5,95x. Il costo è misurato in istruzioni eseguite, non in tempo, perché i programmi durano pochi millisecondi (capitolo 11.1).

10. Bit-flip nei registri. Senza protezione il 2,11% dei guasti produce una SDC; con EDDI nessuna su 2000 (con il 95% di confidenza meno dello 0,19%, più di dieci volte meno); CFCSS e RASM da soli poco (0,75% e 0,80%); EDDI con una tecnica del flusso 1 o 4 casi su 2000, che non si distinguono da EDDI da solo (capitolo 11.2).

11. Salti invertiti. Senza protezione il 79,3% è una SDC. EDDI e CFCSS rilevano quasi solo i salti dei loro controlli (76,3% e 62,4% dei salti eseguiti) e lasciano il 18,75% e il 29,05%; RASM rileva il 99,3%, e le sue 14 SDC vengono tutte dal salto che il back-end crea per la `select` di `load_at`; EDDI+RASM non ne lascia nessuna (capitolo 11.3). È la slide in cui è più facile ricevere domande tecniche: rileggete bene il capitolo 11.3.

12. Bug di RASM: il sintomo. La slide più importante. EDDI+RASM lasciava il 9,70% di SDC, contro lo 0,70% di RASM da solo: un controsenso. Con `sdc_sites.py` si vede che tutte le 194 SDC erano salti verso blocchi che cominciano scrivendo una costante nella firma: i `VerificationBB` di EDDI, dove RASM reimpostava la firma invece di controllarla (capitolo 8.3).

13. Bug di RASM: la correzione. Una riga: l'eccezione vale solo per i blocchi il cui nome comincia con `RASM_`; lo stesso in RACFED. Dopo la correzione: da 194 a 0 SDC, i controlli di RASM rilevano il 67,1% dei salti invertiti invece del 36,4%, e i test di ASPIS danno gli stessi risultati.

16.4 Parte 4: integrazione, limiti, IA (slide 14-17)

14. Integrazione. Due patch (la correzione e i test) e un bug report, pronti ma non inviati. Qui fate le **tre domande**:

1. preferite una sola pull request con le due patch, o la correzione di RASM da sola?
2. apriamo una issue per il bug di EDDI?
3. quattro applicazioni bastano come test di regressione?

Prendete nota delle risposte: decidono cosa fare dopo la chiamata (capitolo 12).

15. Limiti. Gli stessi della sezione 7 del report: solo registri generali e salti, intervalli larghi per la singola applicazione, costo in istruzioni e non in tempo, salti del back-end invisibili alle tecniche sull'IR, bug di EDDI aggirato e non corretto. Dirli voi prima che ve li chiedano fa buona impressione.

16. Intelligenza artificiale. Le regole le hanno date loro: si può usare l'IA anche per tutto il progetto, purché lo si dichiari nel report. Ditelo in modo diretto: abbiamo usato Claude tramite Claude Code per il codice, gli esperimenti, il debugging e il report; noi abbiamo definito obiettivi e perimetro, seguito il lavoro e controllato i risultati. Poi mostrate, nelle risposte, di conoscere ogni parte: è lo scopo di questa guida.

17. Domande. Chiudere chiedendo se vogliono altro nel report prima della consegna.

16.5 Dopo la chiamata

- Aggiornare il report con quello che chiedono (il sorgente è `ASPIS-Project/report/report.md`, poi `python3 report/build.py`, capitolo 13).
- Se vogliono le patch: fork di `HEAPLab/ASPIS`, `git am` delle patch su un branch, pull request (capitolo 12). Se vogliono la issue: aprirla con il testo di `aspis-integration/bug_report.md`.
- Se chiedono altre applicazioni, la procedura è quella del capitolo 7.2: aggiungerle in `testing/tests/c/applications/` e in `tests.toml`, e rilanciare i test.

17. Domande probabili all'orale

Le risposte sono brevi di proposito: sono il nucleo da cui partire, poi si può approfondire con i capitoli indicati.

1. Cos'è ASPIS e a cosa serve? Un insieme di pass di LLVM che trasforma un programma per renderlo capace di rilevare i guasti transitori dell'hardware, duplicando i calcoli (EDDI e varianti) e controllando il flusso di esecuzione con firme (CFCSS, RASM, RACFED). Funziona sull'IR, quindi vale per qualunque linguaggio e architettura supportati da LLVM. (Cap. 4-5)

2. Perché lavorare sull'IR e non sul sorgente o sull'assembly? L'IR è indipendente dal linguaggio e dalla macchina, ed è facile da trasformare con le API di LLVM. Il prezzo è che non si vede quello che fa il back-end: per esempio i salti con cui traduce una `select` (la SDC residua di RASM in `pid_control`). (Cap. 2, 11.3)

3. Perché ASPIS compila a `-O0` e non ottimizza dopo? Perché un ottimizzatore riconoscerebbe che le due copie calcolano la stessa cosa e le fonderebbe, togliendo la protezione. (Cap. 5.4)

4. Cos'è una SDC e perché è l'esito peggiore? Un risultato sbagliato senza nessun segnale d'errore: chi usa il risultato non sa di non potersi fidare. Crash, hang e rilevamenti sono almeno visibili. (Cap. 3.3)

5. Come funziona EDDI? Dove mette i controlli? Duplica ogni istruzione su copie separate dei dati e confronta le copie prima di store, salti condizionali e chiamate, dove un valore sbagliato uscirebbe dal calcolo. Se differiscono chiama `DataCorruption_Handler`. (Cap. 4.1, 5.6)

6. Che differenza c'è tra EDDI, sEDDI, FDSC e REDDI? sEDDI non controlla prima delle store, FDSC controlla solo in alcuni punti, REDDI protegge solo il codice annotato `to_harden`. Costano meno di EDDI ma rilevano meno o più tardi. (Cap. 4.1)

7. Perché EDDI confronta i valori puntati e non i puntatori? Perché le due copie di un puntatore puntano alle due copie del dato, quindi hanno legittimamente indirizzi diversi. Per sapere che tipo leggere serve la libreria TypeDeductionAnalysis, perché in LLVM i puntatori sono opachi. (Cap. 2.6, 5.6)

8. Come funziona CFCSS e perché non rileva i salti invertiti? Ogni blocco ha una firma; all'ingresso si aggiorna la firma di esecuzione con uno XOR della differenza tra la firma del predecessore e quella del blocco, e la si confronta. Un salto invertito porta in un successore lecito, e l'aggiornamento torna giusto: CFCSS controlla che il percorso sia un arco del grafo, non che sia quello scelto dalla condizione. (Cap. 4.3)

9. E RASM perché li rileva? Prima di uscire da un blocco aggiorna la firma con un valore scelto, con una `select`, dalla stessa condizione del salto. Se il salto va dalla parte opposta, la firma corrisponde all'altro successore e il controllo all'ingresso fallisce. (Cap. 4.4)

10. Quali bug avete trovato? Due. (1) Dopo le chiamate di libreria EDDI copiava sempre 8 byte dall'originale alla copia di ogni argomento puntatore, anche dentro le stringhe letterali in memoria di sola lettura: faceva andare in crash la prima versione di tutte e quattro le applicazioni. (2) RASM non controllava la firma nei blocchi `VerificationBB` creati da EDDI, quindi EDDI+RASM perdeva i salti invertiti. Il primo lo abbiamo isolato, segnalato e aggirato; il secondo lo abbiamo corretto. (Cap. 8)

11. Come avete trovato il bug di EDDI? Con i test di compatibilità, che fallivano in 120 combinazioni su 220. Con `gdb`: il crash era sempre una scrittura di 8 byte subito dopo un `printf`, in una stringa letterale (`"kat ct: "`, `"OK"`, `"SUCCESS"`). Poi un programma di cinque righe che lo riproduce, e la lettura di `fixFuncValsPassedByReference` in `EDDI.cpp`. (Cap. 8.2)

12. Perché non avete corretto il bug di EDDI? Perché la correzione vera richiede di riscrivere quella funzione: copiare la dimensione giusta dell'oggetto puntato, saltare la memoria di sola lettura e gli argomenti che la funzione chiamata non può modificare. È un intervento complesso nel codice C++ di EDDI, che chi mantiene ASPIS può fare meglio di noi. Lo abbiamo descritto in `bug_report.md` con il riproduttore e una proposta di correzione. (Cap. 8.2)

13. Cambiare le applicazioni per far passare i test non è un trucco? No, se lo si dichiara: il bug resta documentato e riproducibile (`bug_report.md`, `string_literal.c`), la prima versione delle applicazioni è nella storia di git e il log `harness_master_first_version.log.gz` mostra le 120 combinazioni che fallivano. Le applicazioni modificate stampano lo stesso identico output. Il report lo dice tra i limiti: quella parte di EDDI non è provata dalle nostre applicazioni. (Cap. 7.3, 8.2)

14. Come avete trovato il bug di RASM, visto che nessun test fallisce? Dai numeri: EDDI+RASM aveva più SDC di RASM da solo (9,70% contro 0,70%), il che non ha senso. Con `sdc_sites.py` abbiamo visto che tutte le 194 SDC erano su salti verso blocchi che cominciavano scrivendo una costante nella firma, cioè blocchi in cui RASM reimpostava la firma senza controllarla. Erano i `VerificationBB` di EDDI, esclusi per errore da un controllo sul nome del blocco. (Cap. 8.3)

15. Come avete verificato che la correzione non rompa nulla? Con il sistema di test ufficiale di ASPIS: sui 48 test esistenti i risultati con e senza le patch sono identici (2453 superati, 33 falliti, 154 saltati); le nostre applicazioni passano in tutte le 220 combinazioni, sia su `master` sia con le patch. (Cap. 7.3)

16. Perché 33 test falliscono anche su `master`? Sono tutti di `cpp_complex`: il programma di riferimento non compila con la libreria C++ della nostra macchina (usa `std::powf`, che libstdc++ 13 non dichiara). Non dipende da ASPIS. (Cap. 7.3)

17. Come iniettate i guasti? Con gdb. Una prima esecuzione sotto valgrind (`callgrind`) conta quante volte viene eseguita ogni istruzione; poi ogni esperimento sceglie a caso un'esecuzione di un'istruzione (cioè un istante), e uno script per gdb mette un breakpoint su quell'istruzione con un *ignore count*, così il programma si ferma lì; inverte un bit di un registro (`set $rax = ...`) o manda il salto dall'altra parte (cambiando `$pc`), lo lascia proseguire, e lo script della campagna classifica l'esito confrontando l'output con quello di riferimento. (Cap. 9)

18. Perché non avete scritto un vostro iniettore? Perché gdb fa già tutto quello che serve: fermare il programma su un'istruzione precisa, cambiare i registri e farlo ripartire. Un iniettore scritto da noi con `ptrace` (c'era nelle versioni precedenti) è più veloce, ma molto più lungo e difficile da verificare. Il prezzo di gdb è il tempo: circa 0,4 secondi per esperimento invece di 0,06, quindi abbiamo fatto 500 guasti per combinazione invece di 1000. (Cap. 9.5)

19. Perché gli indirizzi sono gli stessi in tutte le esecuzioni? Perché gdb disattiva la randomizzazione degli indirizzi (ASLR): il programma viene caricato sempre alla stessa base, e lo script calcola la base confrontando l'indirizzo di `main` prima e dopo l'avvio. (Cap. 9.1, 9.3)

20. Perché scegliere l'istante uniformemente tra le istruzioni eseguite? Perché un guasto transitorio può arrivare in qualunque momento: scegliere un'esecuzione a caso di un'istruzione a caso (pesata con il numero di esecuzioni) equivale a scegliere un istante a caso del tempo di esecuzione. (Cap. 9.2)

21. Quanti esperimenti e quanto sono affidabili i numeri? 24.000 (4 applicazioni × 6 configurazioni × 2 modelli × 500), più 4000 di EDDI+RASM senza la correzione di RASM. Ogni percentuale ha un intervallo di confidenza al 95% di Wilson: per esempio la SDC della `baseline` con i bit-flip nei registri è 2,11%, con intervallo 1,56-2,83%; con EDDI, 0 SDC su 2000, cioè meno dello 0,19%. (Cap. 10.3)

22. Perché l'intervallo di Wilson e non quello classico? Perché quello classico funziona male con percentuali vicine a zero (con zero casi dà un intervallo nullo); Wilson resta corretto. (Cap. 10.3)

23. Quanto costano le protezioni? EDDI esegue 3,74 volte le istruzioni e ha il codice 9,27 volte più grande; CFCSS 1,65x, RASM 2,33x; EDDI+RASM circa 6x (5,95x). (Cap. 11.1)

24. Perché il codice cresce più dell'esecuzione? Perché ogni funzione esiste in più versioni (`f`, `f_ret`, `f_dup`, `f_ret_dup`), anche quelle non più usate restano nell'eseguibile, ogni controllo aggiunge blocchi, e a `-O0` tutto passa dallo stack. (Cap. 11.1)

25. Perché EDDI "rileva" il 77% dei salti invertiti, se non vede i salti? Perché il 76% dei salti condizionali eseguiti da un programma con EDDI sono i salti dei suoi controlli: invertirne uno manda il programma nel blocco d'errore. Classificando i salti colpiti, 1517 dei 1543 rilevamenti erano salti dei controlli; gli altri 26 sono salti del programma nelle funzioni duplicate, come la `select` di `load_at`, che con EDDI diventa due salti, uno per copia. Lo stesso per CFCSS (62%), dove tutti i rilevamenti sono salti dei controlli. (Cap. 11.3)

26. Qual è la configurazione migliore? Dipende da cosa si teme e da quanto si può pagare: contro i guasti nei dati serve EDDI, contro i salti sbagliati serve RASM; insieme non lasciano quasi nessuna SDC (4 su 2000 per i registri, che non si distinguono dallo 0 di EDDI da solo, e 0 su 2000 per i salti), ma costano circa 6 volte le istruzioni. (Cap. 11.6)

27. Quali sono i limiti del vostro lavoro? Solo registri generali e salti (niente registri in virgola mobile, memoria, guasti multipli); 500 guasti per applicazione, quindi intervalli larghi sulla singola applicazione; costo misurato in istruzioni e non in tempo; le tecniche sull'IR non vedono i salti creati dal back-end; il bug di EDDI è aggirato e non corretto. (Report, sezione 7)

28. Come avete usato l'intelligenza artificiale? I supervisori l'hanno permesso a patto di dichiararlo. Abbiamo usato Claude (Anthropic), tramite Claude Code, per scrivere il codice, eseguire gli esperimenti, cercare e correggere i bug e scrivere il report; noi abbiamo definito obiettivi e perimetro, seguito il lavoro e controllato i risultati. È dichiarato nella sezione 8 del report. Conviene essere pronti a spiegare ogni parte: è lo scopo di questa guida.

29. La correzione e i test sono già in ASPIS? No: sono pronti come patch (`aspis-integration/patches/`), verificate sul commit `b66a090`, e il bug di EDDI è descritto in `bug_report.md`, ma non è stato inviato ancora nulla. Si possono proporre con una pull request e una issue, dopo averne parlato con i supervisori nella chiamata. (Cap. 12, 16)

30. Quattro applicazioni bastano? È una delle domande da fare ai supervisori (che avevano scritto "once you have identified them, we can discuss whether they are sufficient"). Gli argomenti a favore: coprono quattro settori diversi (crittografia, segnali, compressione, controllo), usano interi, virgola mobile, puntatori, struct e la libreria C, e ognuna viene provata con 55 combinazioni di tecniche. Se ne chiedono altre, aggiungerle è facile (cap. 7.2).

18. Glossario

Termine	Significato
ASLR	*Address Space Layout Randomization*: il sistema operativo carica il programma a indirizzi casuali; gdb la disattiva
basic block	sequenza di istruzioni senza salti interne, con un solo ingresso e una sola uscita
back-end	la parte del compilatore che traduce l'IR in codice macchina
breakpoint	punto in cui il debugger ferma il programma; con un *ignore count* N viene ignorato per N passaggi
callgrind	lo strumento di valgrind che conta quante volte viene eseguita ogni istruzione
CFC	*control-flow checking*, controllo del flusso di esecuzione
CFCSS	tecnica CFC a firme con aggiornamento XOR (Oh et al., 2002)
CFG	*control-flow graph*, grafo dei blocchi e dei salti tra loro
DUE	*detected unrecoverable error*: errore rilevato (crash, hang, controllo)
EDDI	*Error Detection by Duplicated Instructions* (Oh et al., 2002)
ErrBB	blocco creato da EDDI (o dalle tecniche CFC) che chiama il gestore d'errore
fault injection	iniezione artificiale di guasti per misurare l'effetto delle protezioni
FDSC	*Full Duplication with Selective Checking*, variante di EDDI
front-end	la parte del compilatore che analizza il sorgente e produce l'IR (per il C: `clang`)
gdb	il debugger di GNU: lo usiamo per fermare il programma e iniettare il guasto
golden run	esecuzione senza guasti, il cui output fa da riferimento
hang	il programma non termina entro il tempo limite
harness	il sistema di test di ASPIS (`testing/test.py`)
ignore count	quante volte gdb deve ignorare un breakpoint prima di fermarsi
IR	*intermediate representation*, il linguaggio intermedio di LLVM
jcc	salto condizionale x86 (`je`, `jne`, `jl`, …)
masked	il guasto non ha avuto effetti visibili
opt	il programma di LLVM che esegue i pass sull'IR
pass	trasformazione o analisi dell'IR
patch	file con un commit di git (diff + messaggio), applicabile con `git am`
ptrace	chiamata di sistema con cui un processo controlla un altro: è quella che usa gdb
RACFED	variante di RASM che aggiorna la firma dopo ogni istruzione
RASM	*Random Additive Signature Monitoring* (Vankeirsbilck et al., 2017)
REDDI	variante di EDDI che protegge solo il codice annotato `to_harden`
regressione	una modifica che rompe qualcosa che prima funzionava
riproduttore	programma minimo che mostra un bug
SDC	*silent data corruption*: risultato sbagliato senza segnalazione
sEDDI	*selective EDDI*: non controlla prima delle store
select	istruzione IR `c ? x : y`, senza salti
SEU	*single event upset*: inversione di un bit causata da una particella
SIHFT	*Software-Implemented Hardware Fault Tolerance*
SSA	*static single assignment*: ogni valore dell'IR è assegnato una volta
TDA	TypeDeductionAnalysis, la libreria che deduce i tipi puntati

Termine	Significato
valgrind	esegue un programma su un processore simulato per analizzarlo (con `callgrind` conta le istruzioni)
VerificationBB	blocco creato da EDDI che contiene i confronti tra le copie
Wilson (intervallo di)	intervallo di confidenza per una proporzione, affidabile anche vicino a 0

Appendice. La verifica dei fatti

Prima della consegna è stato ricontrollato tutto quello che affermano il report, questa guida, le slide e i README: ogni numero, ogni comportamento descritto e ogni citazione. Questa appendice dice cosa è stato controllato, con quale fonte, e cosa è stato corretto. La verifica è stata fatta il 31 ottobre 2026 sulla versione finale del progetto (la sola correzione di RASM, i guasti iniettati con gdb).

A.1 Il metodo

- **Numeri.** Ricalcolati da zero dai file dei risultati con uno script indipendente da `analyze.py`, `guida/verifica_numeri.py`, e confrontati con le tabelle del report, i capitoli 7-11 della guida e le slide, con lo stesso arrotondamento (per eccesso sul 5). Lo potete rilanciare voi: `python3 guida/verifica_numeri.py ASPIS-Project` (aggiungendo la cartella di lavoro della campagna ricalcola anche le parti che richiedono gli eseguibili).
- **Comportamenti.** Rifatti davvero: il riproduttore compilato e lanciato, le patch applicate su un clone pulito, una parte degli esperimenti ripetuta, gli eseguibili disassemblati, il metodo di iniezione provato su casi noti.
- **Affermazioni sul codice di ASPIS.** Lette nei sorgenti del commit `b66a090`.
- **Citazioni e fonti.** Confrontate con i documenti originali.

Legenda: ✓ confermato; **corretto** = era sbagliato ed è stato corretto (i dettagli in A.9).

A.2 La fault injection

Affermazione	Fonte del controllo	Esito
24.000 esperimenti più 4000 prima della correzione; 11 bit-flip non iniettati (9 in `fft`, che chiama spesso la libreria matematica), esclusi dai conteggi	righe di `raw.csv.gz` e `before_fix/raw.csv.gz`, `outcomes.csv`	✓
SDC dei bit-flip nei registri: 2,11% (baseline), 0,00% (EDDI), 0,75% (CFCSS), 0,80% (RASM), 0,05% e 0,20% (EDDI con CFCSS e con RASM), con gli intervalli di Wilson delle tabelle	`outcomes.csv`, ricalcolo	✓
SDC dei salti invertiti: 79,30%, 18,75%, 29,05%, 0,70%, 7,70%, 0,00%	`outcomes.csv`, ricalcolo	✓
EDDI+RASM prima della correzione: 194 SDC (9,70%) e 36,4% rilevati dalle firme; dopo: 0 SDC e 67,1%	`before_fix/outcomes.csv`	✓
Le tabelle per applicazione (report 5.5, guida 11.5)	`outcomes.csv`, ricalcolo	✓
Crash della baseline: 286, di cui rsp 47% e rbp 42% (insieme 89%)	`raw.csv.gz`	✓
Costo: EDDI 3,74x istruzioni e 9,27x codice, CFCSS 1,65x, RASM 2,33x, EDDI+CFCSS 5,46x, EDDI+RASM 5,95x; 1283 controlli; EDDI tra 3,56x e 4,12x	`metrics.csv`, media geometrica	✓
Un esperimento dura in media circa 0,4 secondi	`time_ms`: media 376 ms, mediana 283 ms	✓
Salti dei controlli: 76,3% con EDDI, 62,4% con CFCSS, 67,4% con RASM	`check_jumps.py` sugli eseguibili della campagna	✓
EDDI e CFCSS rilevano quasi solo i salti dei loro controlli	classificazione dei salti colpiti: tutti i 1249 rilevamenti di CFCSS e 1517 dei 1543 di EDDI sono salti dei controlli	✓
Le 14 SDC di RASM vengono tutte da un solo salto di `load_at`	`sdc_sites.py`: tutte al `jne` di `0x1de8`; nel disassemblato la funzione (`__unnamed_8`) confronta con `0x4b0` = 1200 e `0x514` = 1300	✓
Con EDDI la `select` di `load_at` diventa due salti, uno per copia, e la loro inversione viene rilevata	disassemblato di `pid_control` con EDDI: 11 rilevamenti ai due salti	✓
Le 194 SDC prima della correzione sono tutte su salti verso blocchi che scrivono una costante nella firma, in tutti i casi con entrambi i successori	disassemblato degli eseguibili senza correzione	✓
Gli esperimenti sono ripetibili	30 esperimenti già fatti, rilanciati con gli stessi parametri: stesso esito, codice d'uscita e segnale in 30 casi su 30	✓
Ricompilando, l'eseguibile di EDDI può cambiare un poco	tre compilazioni di `aes128` con EDDI: stessa dimensione e stessi 284 controlli, ma codice diverso	✓

Il metodo di iniezione. Prima di usare gdb e valgrind abbiamo verificato che facciano quello che diciamo:

Affermazione	Fonte del controllo	Esito
valgrind conta le istruzioni eseguite correttamente	sui 24 programmi della campagna precedente, i totali di valgrind e del nostro vecchio iniettore (che eseguiva il programma un'istruzione alla volta) differiscono sempre di 12 istruzioni, quelle di `_init`, `_fini` e di un salto della PLT, che valgrind non attribuisce al programma; istruzione per istruzione i conteggi coincidono, tranne per la PLT, che valgrind conta insieme alle `call`	✓
Un breakpoint con *ignore count* `k`-1 ferma il programma alla `k`-esima esecuzione	sessione gdb del capitolo 9.3: con `ignore 1 1807` gdb riporta "breakpoint already hit 1808 times"	✓
Gli *hang* vengono rilevati	un programma con un ciclo infinito: ucciso dal processo di guardia (`SIGKILL`) e classificato come *hang*	✓
I risultati non dipendono dall'iniettore	per tutti i 14 tassi di SDC (6 configurazioni × 2 modelli, più EDDI+RASM senza correzione) gli intervalli al 95% della campagna con gdb e di quella precedente, fatta con il nostro iniettore, si sovrappongono	✓

A.3 I test di compatibilità

Affermazione	Fonte del controllo	Esito
52 test × 55 combinazioni = 2860 compilazioni per esecuzione	i tre log	✓
Prima versione su `master`: 100 combinazioni su 220; versione finale: 220 su 220, su `master` e con le patch	i tre log	✓
Test esistenti: 2453 superati, 33 falliti, 154 saltati, identici nei tre log	i tre log	✓
I 33 falliti sono tutti di `cpp_complex`	i log: 55 combinazioni meno le 22 della sua `black_list`	✓
I 154 saltati sono le combinazioni delle `black_list`	i log: 14 coppie test-tecnica del flusso × 11 opzioni dei dati	✓
Ogni esecuzione dura circa 9 minuti	ultima riga dei log: da 8 min 55 s a 9 min 13 s	✓
11 opzioni dei dati × 5 del flusso, riferimento compilato con `clang++`, limite di 60 s	`testing/test.py`	✓

A.4 Le applicazioni

Affermazione	Fonte del controllo	Esito
329, 186, 225 e 159 righe	`wc -l`	✓
645.656, 367.747, 170.962 e 477.379 istruzioni eseguite senza protezione	`metrics.csv` (conteggio di valgrind)	✓
La versione finale stampa esattamente lo stesso output della prima	le due versioni compilate con `gcc` e confrontate byte per byte	✓
Terminano con `SUCCESS` e codice 0; sono valide anche in C++; licenza MIT	esecuzione, `g++ -x c++`, intestazioni SPDX	✓
Le copie nella patch 0002 sono uguali a `benchmarks/`	confronto dei file	✓

A.5 Il bug di EDDI

Affermazione	Fonte del controllo	Esito
Il riproduttore va in crash su `b66a090` con EDDI, sEDDI e FDSC, con qualunque tecnica del flusso, e stampa `hello` senza duplicazione	compilato e lanciato: *segmentation fault* (codice 139) con `--eddi`, `--seddi`, `--fdsc` e con le 4 varianti di EDDI per le 5 tecniche del flusso; `hello` con `--no-dup`	✓
Il comando del README del riproduttore funziona come scritto	eseguito tale e quale	✓
`fixFuncValsPassedByReference` legge un valore della dimensione di un puntatore dall'originale e lo scrive nella copia, per ogni operando con una copia	`passes/EDDI.cpp`	✓
Le nostre patch non correggono questo bug	il riproduttore va in crash anche con le patch	✓

A.6 La correzione di RASM e le patch

Affermazione	Fonte del controllo	Esito
Le due patch si applicano con `git am` su un clone pulito di `b66a090`	applicate	✓
Il risultato è l'ASPIS usato per tutti gli esperimenti	confronto di `passes/` e `testing/`	✓
La correzione cambia una riga in `RASM.cpp` e una in `RACFED.cpp`, più un commento	la patch 0001	✓
`EDDI.cpp` ha circa 2400 righe; la seconda versione vi aggiungeva circa 160 righe	2441 righe; le due patch della seconda versione: 66 + 92 righe aggiunte	✓
DuplicateGlobals tiene coerenti le copie delle globali scritte da funzioni non duplicate; il passo `strip` toglie i nomi delle funzioni `static`	`passes/DuplicateGlobals.cpp`, `aspis.sh`	✓

A.7 Fonti e citazioni

Affermazione	Fonte del controllo	Esito
Le frasi dei supervisori (uso dell'IA, chiamata, report)	`Buongiorno,.txt`	✓ (corretta una parafrasi nel capitolo 1)
Articolo di ASPIS: ACM TODAES 29(5), articolo 91, 2024, doi 10.1145/3660524	il link nell'e-mail dei supervisori	✓
EDDI e CFCSS: Oh, Shirvani, McCluskey, IEEE Transactions on Reliability 51(1), 2002; RASM: Vankeirsbilck et al., IEEE Transactions on Reliability 66(4), 2017	riferimenti bibliografici	✓
LLVM 21.1.8 estratto occupa circa 11 GB	`du`	✓

A.8 I tempi

La campagna principale è durata circa 38 minuti su 4 core (pochi secondi per compilazioni e golden run, il resto esperimenti, dai tempi del suo log); quella senza correzione circa 6 minuti. Report e guida dicono "circa 40 minuti" per la campagna principale. La versione precedente, con il nostro iniettore, ne richiedeva circa 18 per il doppio degli esperimenti: gdb è circa sei volte più lento per esperimento. I tempi dipendono comunque dalla macchina.

A.9 Cosa è stato corretto

Nella verifica di questa versione:

- **report**: il numero di righe di `gdb_inject.py` (una prima stesura diceva “circa 80, metà commenti”: sono 72, di cui 45 di codice) e la didascalia della tabella delle applicazioni (le istruzioni le conta valgrind, non più un profiler nostro);
- **guida, capitolo 9**: una prima stesura diceva che con un profilo corretto nessun esperimento resta “non iniettato”; invece 11 lo sono stati, per come valgrind conta le chiamate attraverso la PLT, e ora è spiegato;
- **guida, capitolo 17**: una risposta attribuiva a EDDI la percentuale della baseline, e un’altra diceva che EDDI+RASM lascia “quasi nessuna” SDC sui registri senza dire che non si distingue da EDDI da solo; corrette.

Nella verifica della versione precedente (commit `0609127`) erano stati corretti: la durata della campagna nel report e nella guida; i capitoli 11 e 17 della guida, che avevano ancora numeri vecchi; i rimandi al capitolo 8.4; il paragrafo sui risultati per applicazione; una parafrasi dei supervisori e la descrizione di un costo nel capitolo 1; i README di `aspis-integration/` e `results/`; la descrizione di FuncRetToRef nelle slide. Prima ancora: la citazione dei supervisori nel report (ora testuale), le descrizioni di FDSC e REDDI, il numero di righe di `EDDI.cpp`, la dimensione di LLVM e l’arrotondamento delle tabelle (ora per eccesso sul 5: 35,05% diventa 35,1%).

A.10 Cosa resta fuori

- La dimensione dello scaricamento di LLVM (circa 2 GB) non è stata ricontrollata in questa verifica.
- La stima del lavoro (circa 60-75 ore in due, con il progetto ridotto) è una stima, non una misura.
- I numeri della campagna valgono per questi eseguibili: ricompilando, il codice prodotto da EDDI cambia un poco (A.2) e i risultati possono variare di qualche caso.
- Sotto gdb il programma non gira esattamente come da solo: gdb aggiunge all’ambiente le variabili `LINES` e `COLUMNS`, e lo stack comincia 64 byte più in basso. Questo può cambiare di poco quali indirizzi sbagliati portano a un crash (nella campagna con gdb i crash della baseline sono il 14,3%, nella precedente il 12,0%), ma non il confronto tra le configurazioni, che girano tutte sotto gdb.